НГТУ НЭТИ

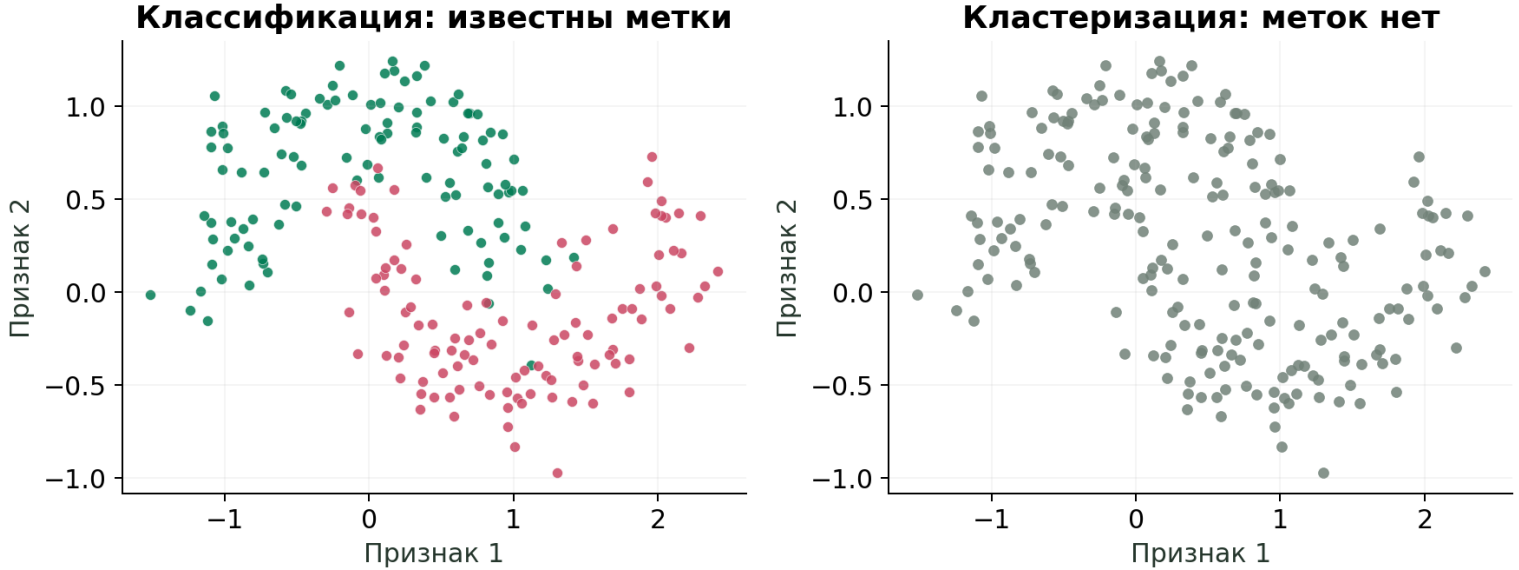
Новосибирский государственный технический университет НЭТИ

Классификация. Нейронные сети. Кластеризация



Обучение с учителем и без учителя

С учителем: известны объекты X и метки классов Y. Модель учится предсказывать Y для новых объектов.
Без учителя: известны только X. Алгоритм ищет структуру в выбранном пространстве признаков.



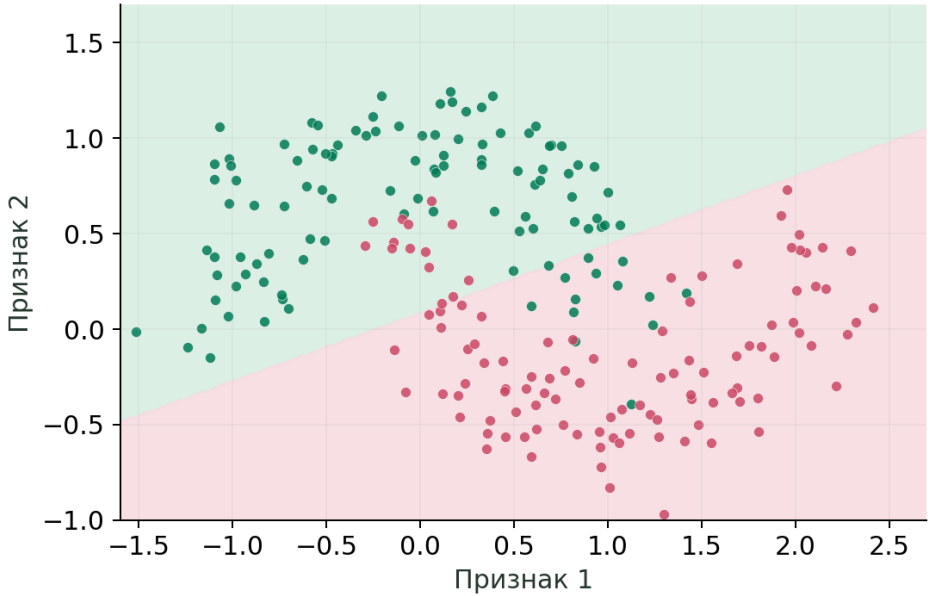
Класс и кластер

Характеристика	Классификация	Кластеризация
Вход	X и метки Y	X
Выход	Известный класс	Группа похожих объектов
Пример	Спам / обычное письмо	Группы похожих писем
Проверка	Сравнение с Y	Структура кластеров
Смысл	Задан заранее	Требует интерпретации

Задача классификации

Объект $x = (x_1, \ldots, x_p)$ описывают признаками.
Цель: построить $f\theta(x)$, которая выбирает класс из конечного множества.
θ — это параметры модели (веса и смещения), которые подбираются при обучении.
Вероятностная модель сначала оценивает $P(y = c \mid x)$ — то, что модель учится предсказывать, а затем принимает решение.

$$\hat{y} = \operatorname{argmax}_c P(y = c \mid x)$$



Виды задач классификации

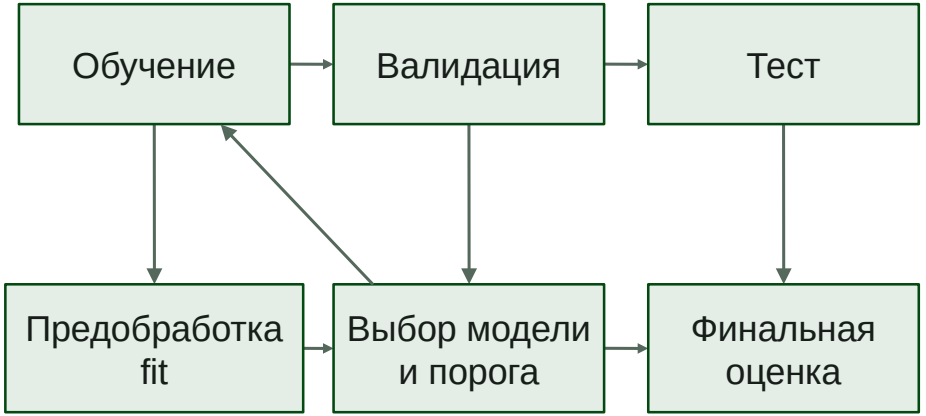
Тип	Ответ для объекта	Пример
Бинарная	Один из двух классов	Спам / не спам
Многоклассовая	Один из C классов	Цифры 0–9
Многометочная	Несколько меток сразу	Темы статьи
Многоцелевая	Несколько целевых переменных	Тип и состояние изделия

Источник: MC · ссылки в заметках

Рабочий процесс классификации

1. Определить классы и цену ошибок.
2. Разделить данные на обучение, валидацию и тест.
3. Предобработка на train (fit), а на val/test только применяем (transform).
4. Сравнить модели и выбрать порог на валидации.
5. Один раз оценить итоговую модель на тесте.

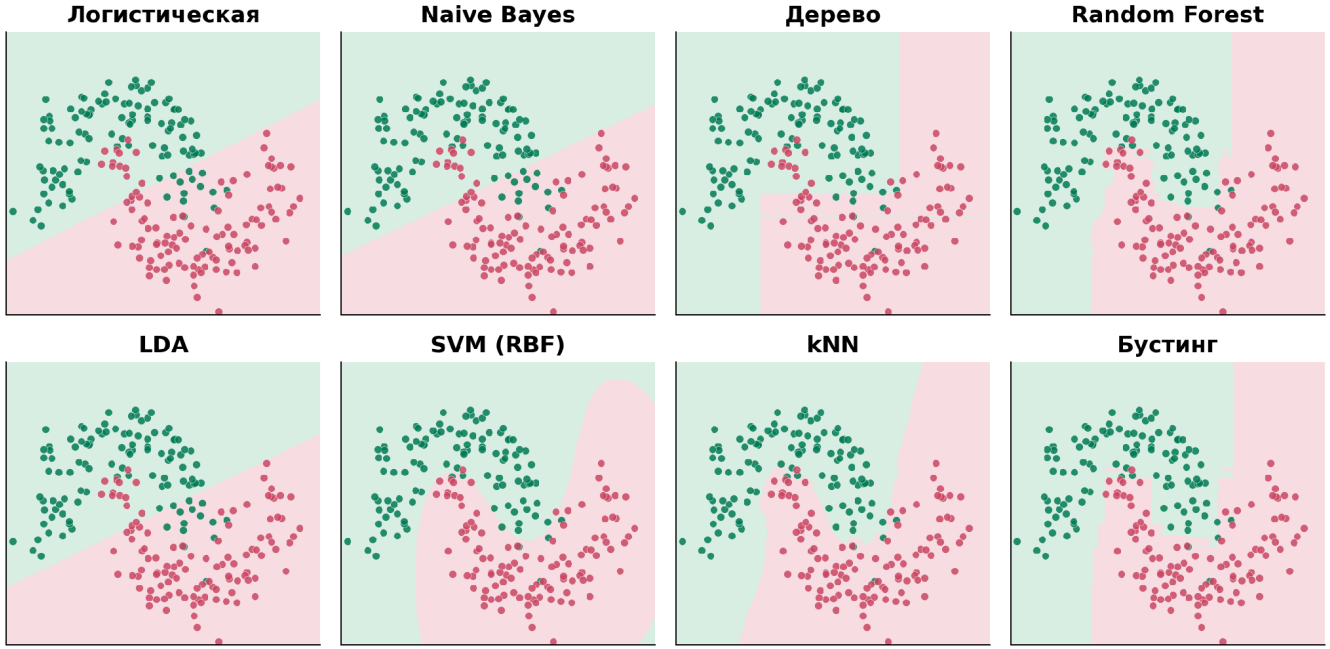
Важно помнить про кросс-валидацию и стратификацию (баланс классов)



Алгоритмы классификации

Линейные: логистическая регрессия, LDA.
Геометрические: SVM и kNN.
Ансамблевые алгоритмы.

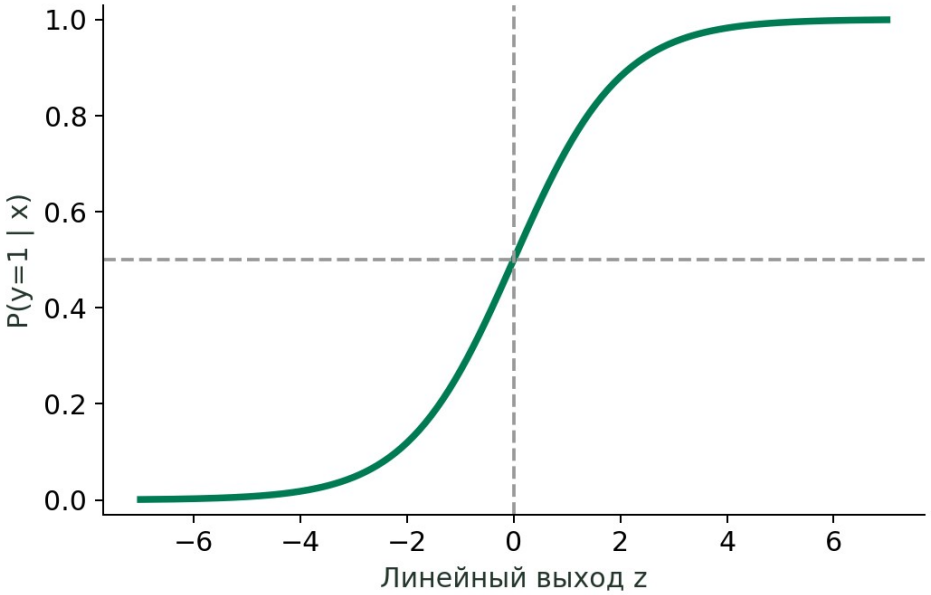
Вероятностные: наивный Байес.
Деревья решений.
Нейронные сети.



Логистическая регрессия

Линейная комбинация признаков задаёт логит z — это логарифм отношения шансов: log(p / (1-p)).
Логистическая регрессия моделирует логит как линейную функцию: $z = w^T x + b$
Sigmoid преобразует z в вероятность класса 1.
Порог τ (по умолчанию 0.5) переводит вероятность в метку класса.

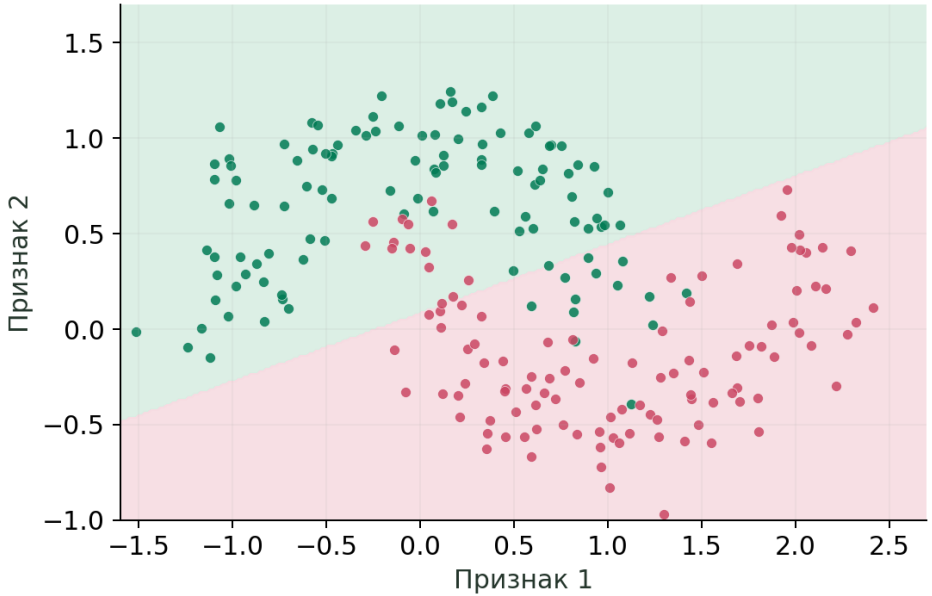
$p = \sigma(z) = 1 / (1 + e^{-z})$



Логистическая регрессия: обучение

Параметры выбирают по log loss (функция бинарной кросс-энтропии).
Регуляризация ограничивает величину весов.
При линейных признаках граница решения линейная (граница решения - гиперплоскость).
Нелинейные признаки могут усложнить ее.

$$L = -(1/n)\,\Sigma_i\,[y_i \ln p_i + (1-y_i)\ln(1-p_i)]$$



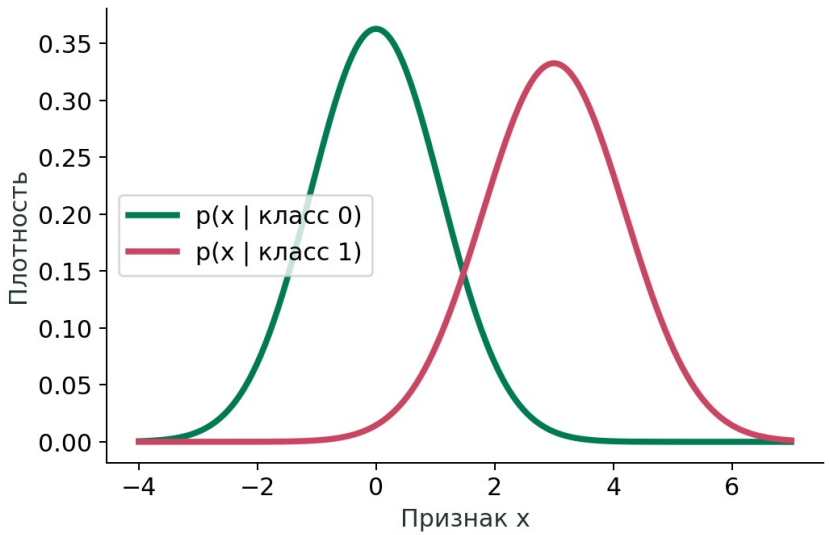
Байесовский классификатор

Выбираем класс с максимальной апостериорной (после наблюдения признаков) вероятностью.
Формула Байеса: **$P(c \mid x) = P(x \mid c) \cdot P(c) / P(x)$**
Компоненты:

- P(c) — априорная (до наблюдения признаков) вероятность класса (частота в выборке).
- P(x | c) — правдоподобие: насколько вероятны признаки x, если объект класса c.
- P(c | x) — апостериорная вероятность (то, что мы ищем).

Решающее правило: **$\hat{y} = \text{argmax}_c \, P(x \mid c) \cdot P(c)$**

P(x) отбрасываем — она одинакова для всех классов



Наивный Байес

Наивный Байес предполагает условную независимость признаков.

Предположение: признаки условно независимы при заданном классе.

GaussianNB: предполагает, что P(x | c) — это нормальное распределение с параметрами μ и σ, которые оцениваются по выборке. Подходит для непрерывных признаков.

MultinomialNB: P(x | c) — это частота события (например, сколько раз слово встречается в спам-письме). Подходит для текстов.

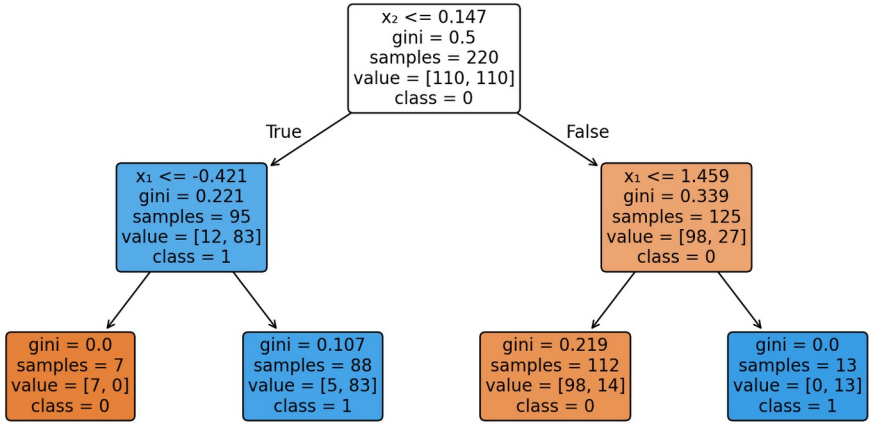
BernoulliNB: P(x | c) — это бинарная вероятность (слово есть / слова нет). Подходит для коротких текстов или бинарных признаков.

Дерево решений

Внутренний узел проверяет условие над признаком, ветви ведут к следующим проверкам, лист хранит класс или распределение классов.

Дерево обучается жадным алгоритмом: перебирает признаки и пороги, выбирает разбиение, максимально уменьшающее неоднородность (Gini или энтропию). Без ограничений переобучается, поэтому ограничивают глубину и минимальное число объектов в листе.

Деревья не требуют масштабирования признаков, интерпретируемы, но одно дерево — слабая модель, поэтому используют ансамбли.

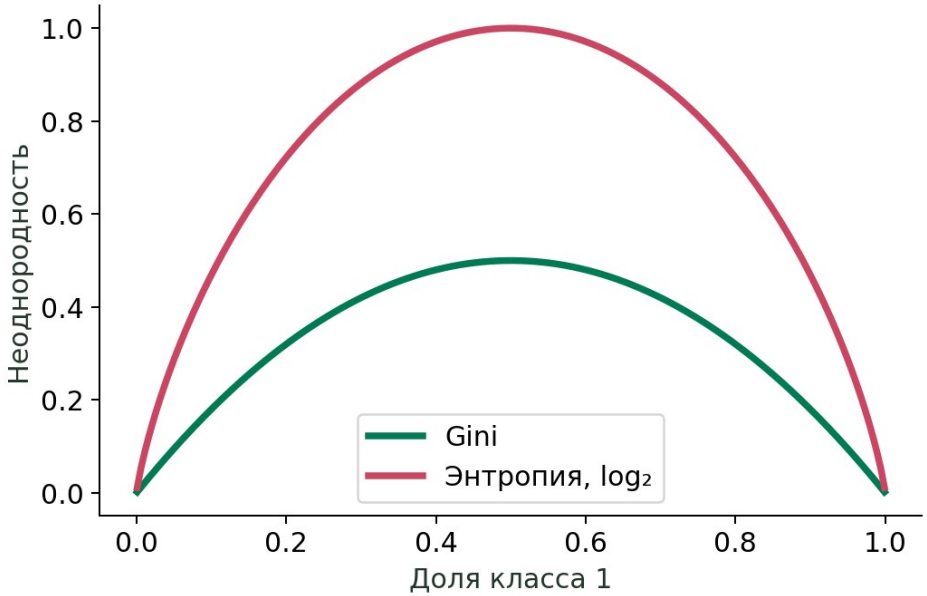


Критерии разбиения дерева

Чистый узел содержит объекты одного класса.

Gini и энтропия измеряют смешанность классов.

Выбирают разбиение с наибольшим уменьшением взвешенной неоднородности.

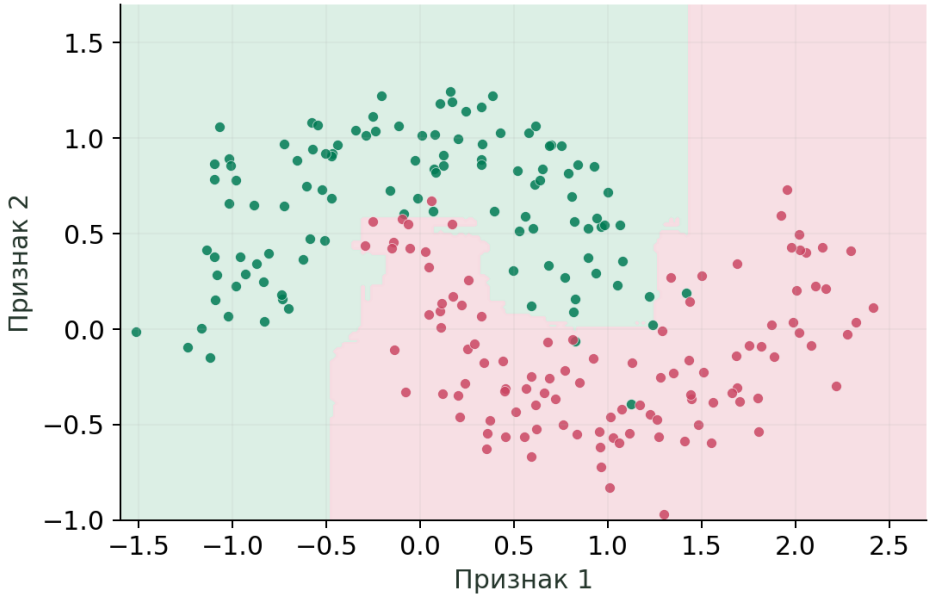


$Gini = 1 - \Sigma_c\ p_c^2$
$H = -\Sigma_c\ p_c\ \log_2 p_c$
где p_c — доля объектов класса c в узле

Случайный лес и крайне рандомизированные деревья

Random Forest обучает много деревьев независимо, каждое на своей bootstrap-выборке и на случайном подмножестве признаков в каждом узле. Предсказания усредняются или выбираются голосованием. Это снижает разброс и борется с переобучением.

Extra Trees идёт дальше: пороги для разбиения выбираются случайно, без поиска лучшего. Обучение быстрее, но смещение выше.



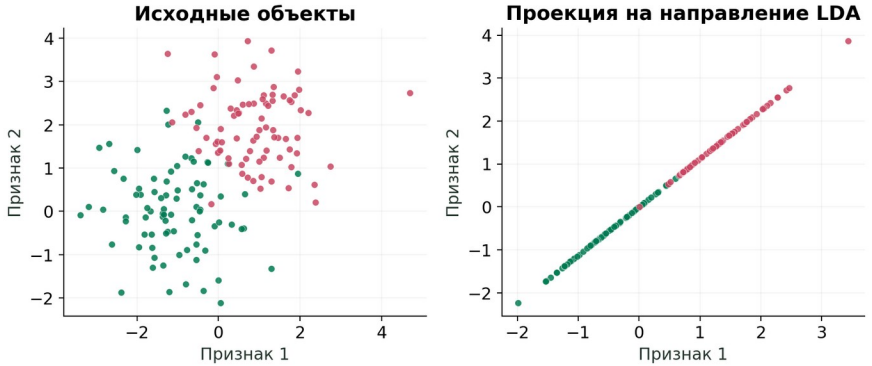
Линейный дискриминантный анализ. LDA

LDA предполагает, что признаки каждого класса распределены нормально, а ковариационные матрицы классов совпадают. Из-за общей ковариации Σ граница решения между классами линейна.

Для нового объекта вычисляется дискриминантная функция δ_c(x) для каждого класса, выбирается класс с максимальным значением.

$$\delta_c(x) = x^T\Sigma^{-1}\mu_c - \tfrac{1}{2}\mu_c^T\Sigma^{-1}\mu_c + \ln \pi_c$$

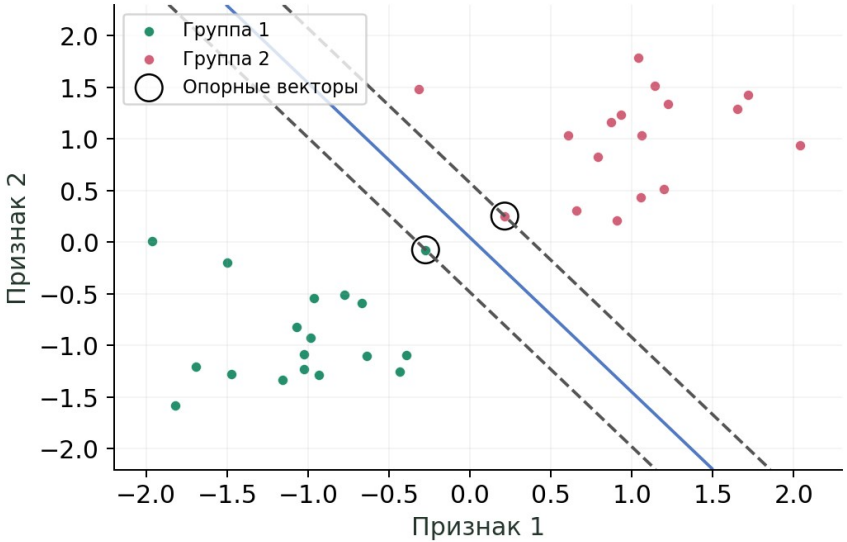
где μ_c — среднее класса, π_c — априорная вероятность класса, Σ — общая ковариационная матрица.



Метод опорных векторов (SVM)

SVM ищет гиперплоскость, разделяющую классы с максимальным зазором. Чем шире зазор, тем устойчивее модель к новым данным. Положение границы определяют опорные векторы — объекты, ближайшие к ней.

Гиперпараметр C управляет штрафом за нарушения зазора: малое C допускает нарушения ради устойчивости, большое C требует строгого разделения и рискует переобучиться.

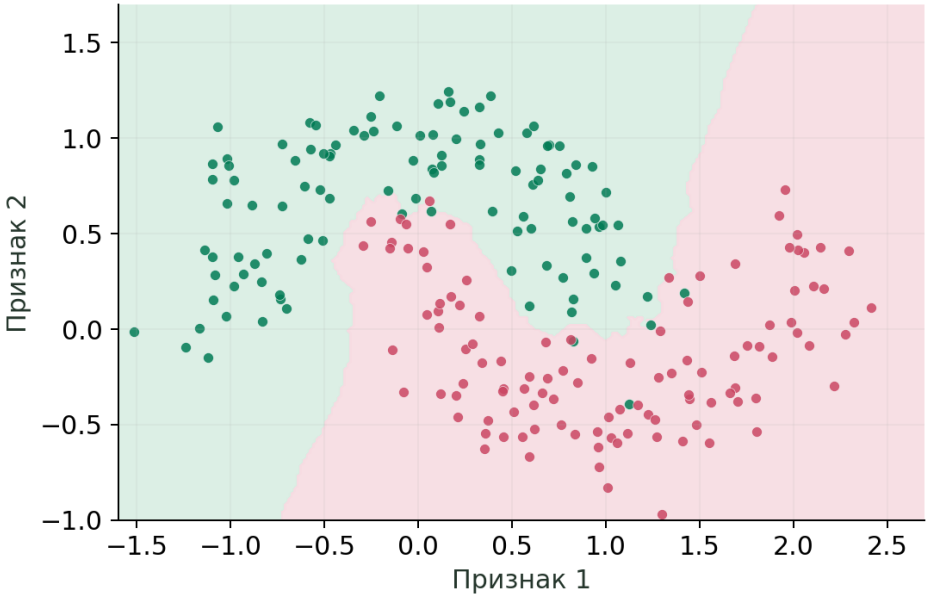


$w^T x + b = 0$, где w — вектор нормали, b — смещение

Метод ближайших соседей: kNN

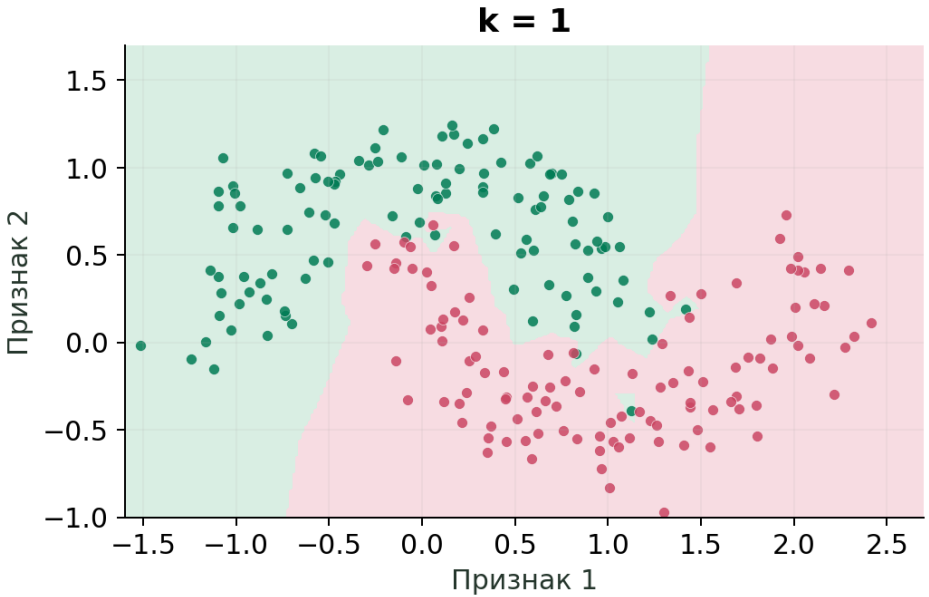
Для нового объекта находим k ближайших обучающих примеров и смотрим на их классы. Класс выбираем по большинству голосов среди соседей, при взвешенном голосовании ближние соседи имеют больший вес.

Важно! kNN использует метки. K-means ищет центры кластеров.



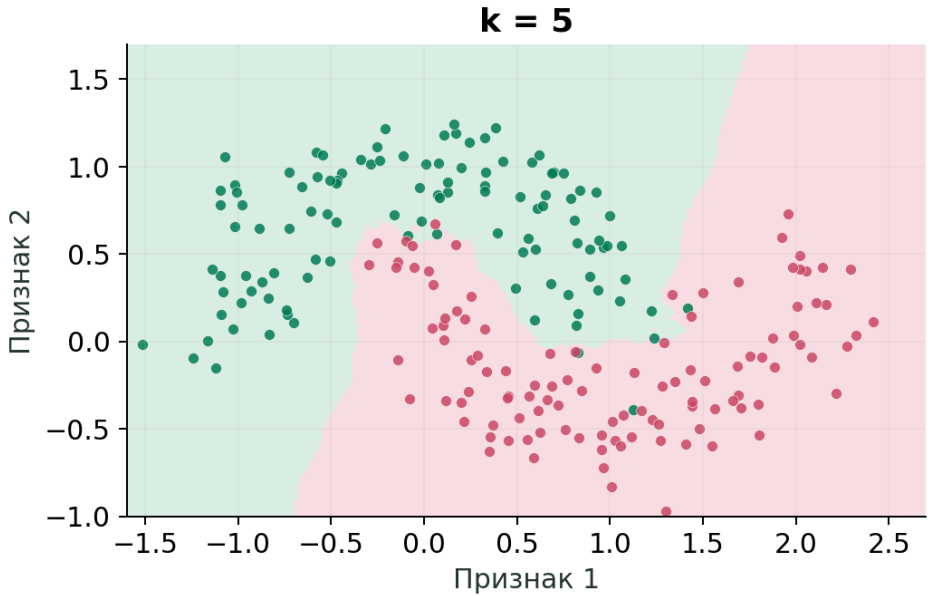
Число соседей и сложность границы

Малое k: локальная, чувствительная к шуму граница.
Большое k: более гладкая граница, возможное недообучение.
k подбирают на валидационной выборке. Перед обучением признаки масштабируют, иначе расстояния будут искажены.



kNN при k=5

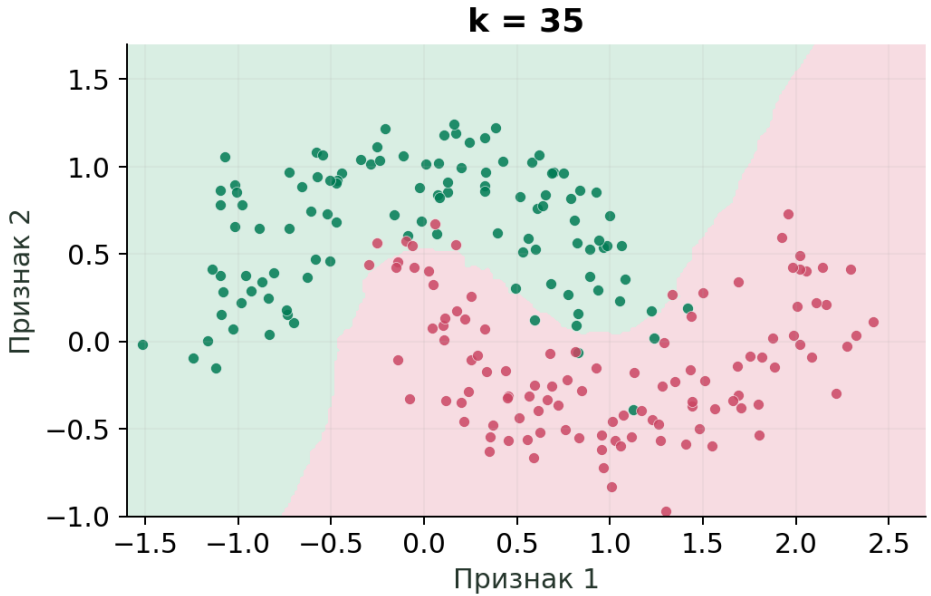
Граница усредняет решение нескольких соседей.



kNN при k=35

Большое число соседей сглаживает границу.

Локальная структура может теряться.



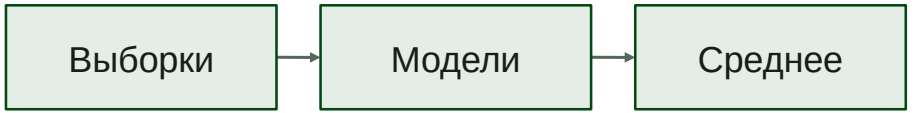
Ансамблевые алгоритмы

Метод	Как обучают	Что объединяют	Что это дает
Бэггинг	Независимо, на bootstrap выборках	Предсказания	Снижает разброс
Бустинг	Последовательно	Корректирующие модели	Снижает смещение
Стекинг	Базовые модели + метамодель (которая обучается на предсказаниях базовых моделей)	Предсказания как признаки	Использует разнообразие моделей

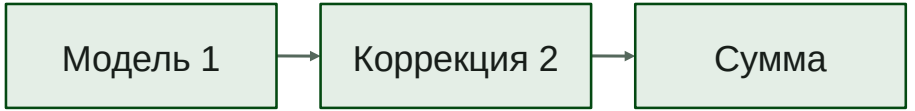
Bootstrap — это новая псевдовыборка, которую создают путем случайного повторного выбора элементов из исходной выборки с возвращением (ресемплинг)

Бэггинг. Бустинг. Стекинг

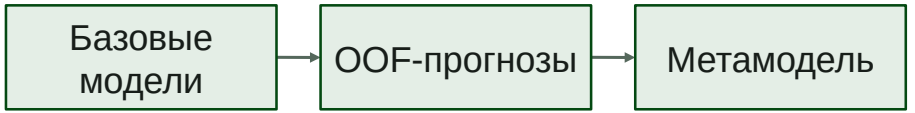
Бэггинг: bootstrap и усреднение.



Бустинг: следующий шаг улучшает текущий ансамбль.



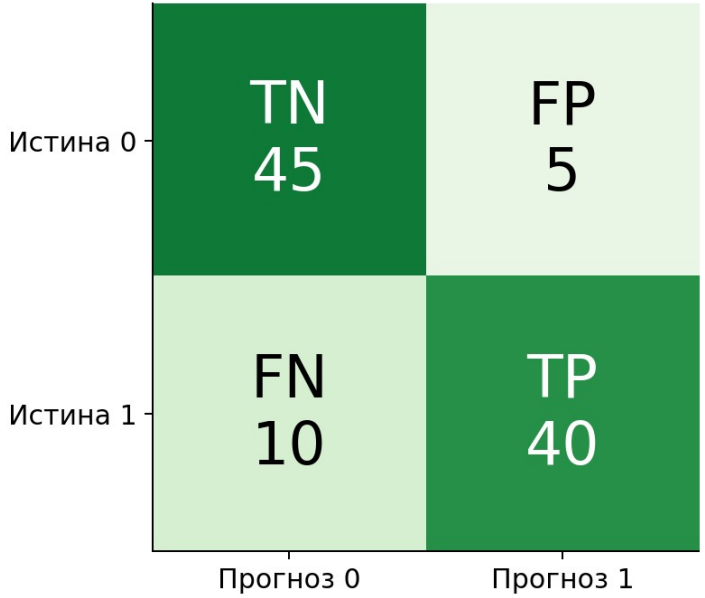
Стекинг: метамодель обучается на out-of-fold прогнозах базовых моделей (предсказания базовых моделей на тех данных, которые не участвовали в их обучении).



Матрица ошибок

Строки: истинный класс.
Столбцы: предсказанный класс.

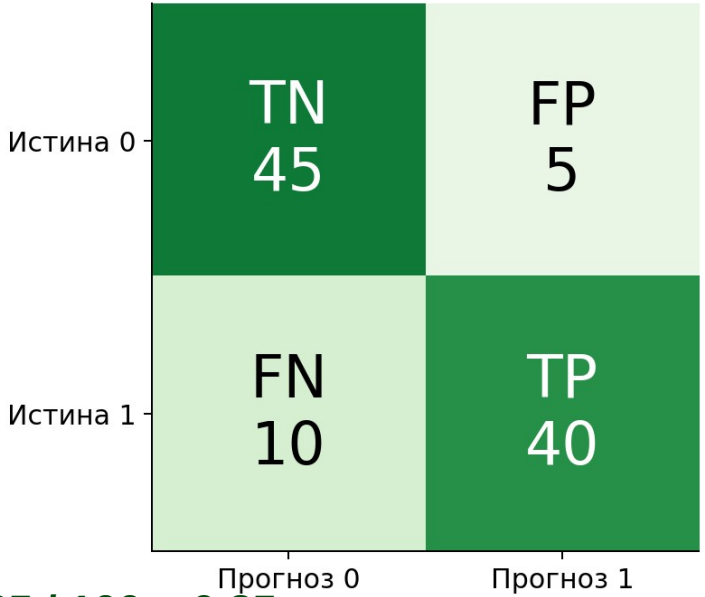
TP (True Positive) — истинно положительный: объект действительно положительный, и модель предсказала положительный.
FP (False Positive) — ложно положительный: объект на самом деле отрицательный, но модель предсказала положительный. Это ошибка I рода (ложная тревога).
FN (False Negative) — ложно отрицательный: объект на самом деле положительный, но модель предсказала отрицательный. Это ошибка II рода (пропуск).
TN (True Negative) — истинно отрицательный: объект действительно отрицательный, и модель предсказала отрицательный.



Accuracy и precision

Accuracy (точность): доля всех верных ответов.

Precision (точность по положительным предсказаниям): доля верных среди положительных прогнозов.

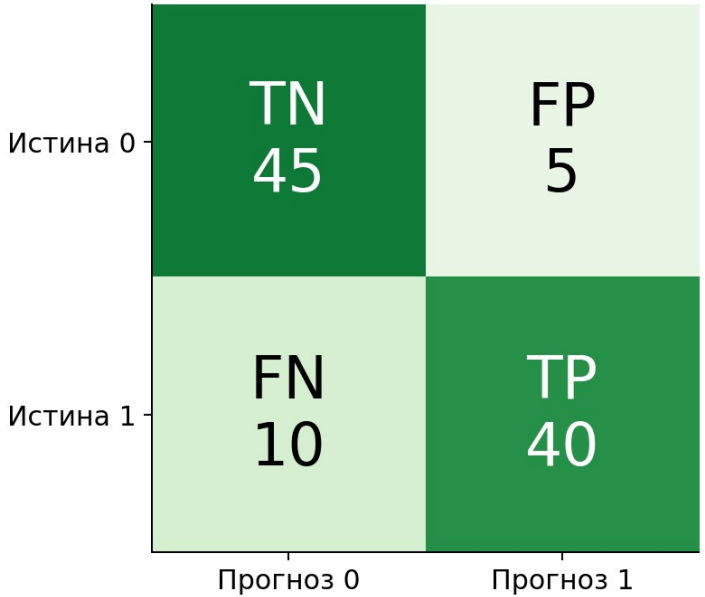


Accuracy = (TP + TN) / (TP + TN + FP + FN) = 85 / 100 = 0.85
Precision = TP / (TP + FP) = 40 / 45 ≈ 0.889

Recall и F1-мера

Recall (полнота): доля найденных положительных объектов.

F1-мера: гармоническое среднее precision и recall.
F1 не учитывает TN и зависит от выбранного положительного класса.



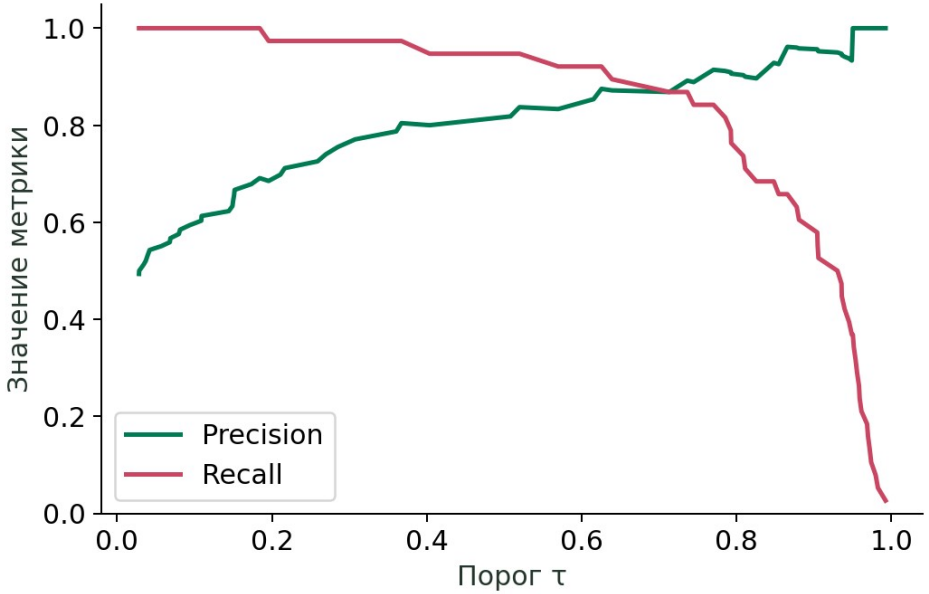
Recall = TP / (TP + FN) = 40 / 50 = 0.80
F1 = 2TP / (2TP + FP + FN) = 80 / 95 ≈ 0.842

Порог классификации

Модель выдает вероятность p. Чтобы получить класс, нужно правило: класс 1, если p ≥ τ. По умолчанию τ = 0.5, но это лишь соглашение, а не закон.

Порог выбирают под задачу. Снижение τ увеличивает recall (ловим больше положительных), но растет число FP (ложных тревог). Повышение τ увеличивает precision, но растёт число FN (пропусков).

Порог подбирают на валидации, исходя из количества ошибок FP и FN.

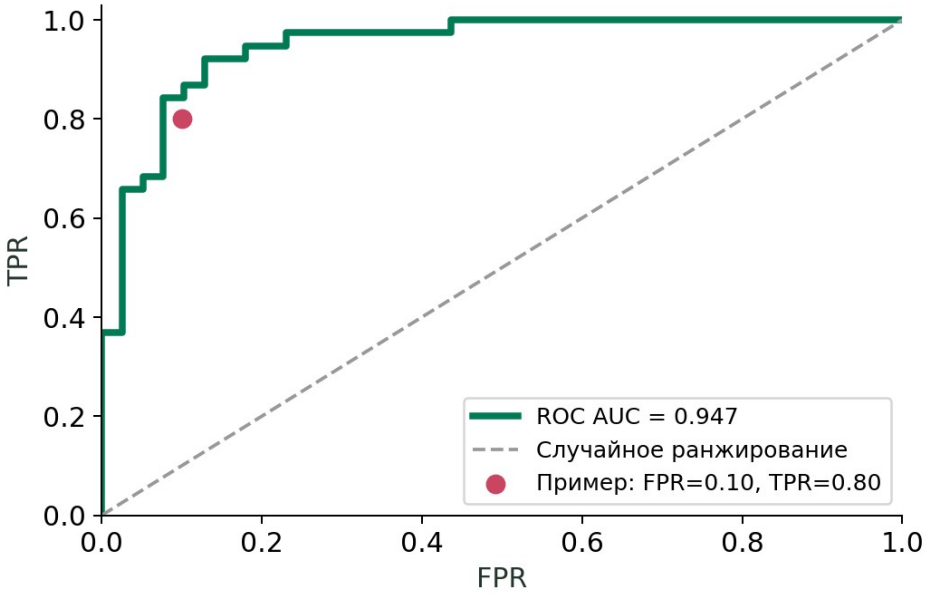


ROC-кривая: FPR и TPR

ROC-кривую строят при изменении порога.

Ось X: FPR, доля ложных срабатываний среди отрицательных.

Ось Y: TPR, доля найденных положительных, то есть recall.

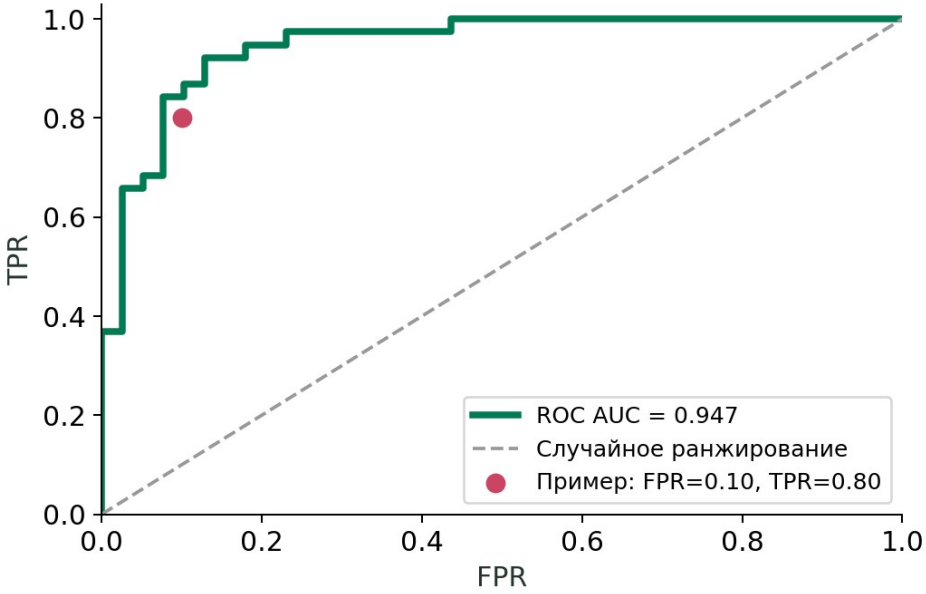


FPR = FP / (FP + TN)
TPR = TP / (TP + FN)

ROC AUC

AUC (площадь под ROC-кривой) измеряет качество ранжирования по всем порогам.
AUC = 0.5: случайное ранжирование в ожидании.

AUC = $\int_0^1$ TPR(u) du, где u = FPR



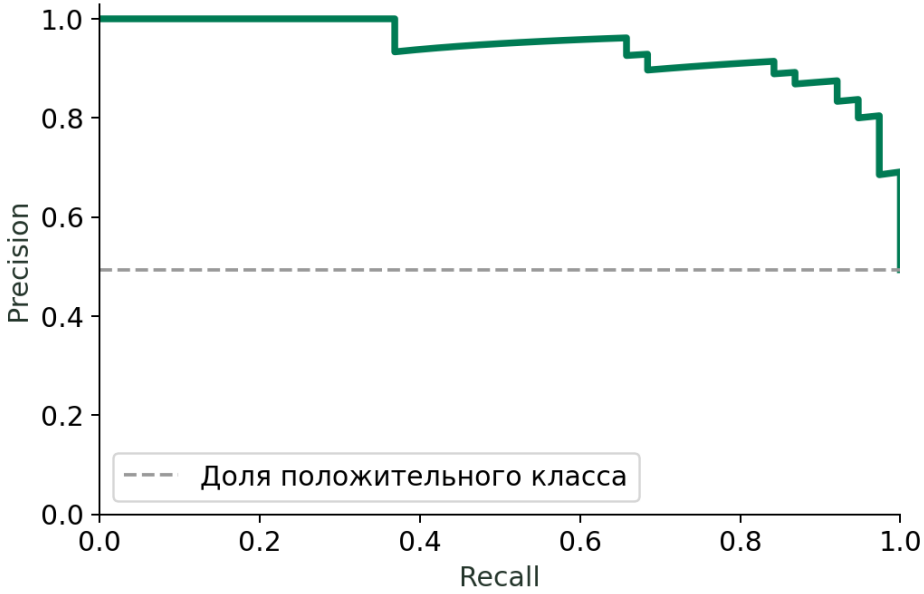
PR-кривая

PR-кривая показывает, как меняются precision и recall при изменении порога. По оси X — recall, по оси Y — precision. Каждая точка — свой порог.

Снижение порога: recall растёт, precision падает. Повышение: precision растёт, recall падает. Идеал — правый верхний угол.

Площадь под кривой (Average Precision, AP) — мера качества. Чем ближе к 1, тем лучше.

PR-кривая важна при дисбалансе классов, когда положительный класс редкий. ROC может выглядеть хорошо, а PR покажет реальную картину.



Микро- и макроусреднение

В многоклассовой задаче метрики считают для каждого класса отдельно (one-vs-rest), а затем усредняют двумя способами.

Macro: сначала считаем метрику для каждого класса, затем берём среднее. Каждый класс имеет одинаковый вес, поэтому редкие классы влияют так же сильно, как частые.

Micro: сначала суммируем TP, FP и FN по всем классам, затем считаем одну общую метрику. Частые классы доминируют.

Macro Precision = (1/C) Σ Precision_c

Macro Recall = (1/C) Σ Recall_c

Macro F1 = (1/C) Σ F1_c

Micro Precision = Σ TP_c / (Σ TP_c + Σ FP_c)

Micro Recall = Σ TP_c / (Σ TP_c + Σ FN_c)

Micro F1 = 2Σ TP_c / (2Σ TP_c + Σ FP_c + Σ FN_c)

Выбор: Macro — когда все классы важны одинаково. Micro — когда важна общая точность по всем объектам.

Усреднение: числовой пример

Истина / прогноз	A	B	C
A	80	10	0
B	3	6	0
C	1	0	0

Класс A: F1 ≈ 0.920. Класс B: F1 = 0.480. Класс C: F1 = 0.
Macro F1 ≈ 0.467. Micro F1 = accuracy = 0.860.

Проверка понимания: классификация

В выборке 990 обычных операций и 10 мошеннических. Модель всегда отвечает «обычная».

Чему равны accuracy и recall мошенничества?

Понятие глубокого обучения

Глубокое обучение — это машинное обучение на многослойных нейронных сетях. Каждый слой преобразует вход и передаёт результат дальше. Первые слои извлекают простые признаки, последние — сложные.

Главное отличие от классического ML: признаки не конструируются вручную, а извлекаются автоматически в процессе обучения.

Глубина — это число последовательных слоев. Чем больше слоёв, тем более абстрактные представления может строить модель.

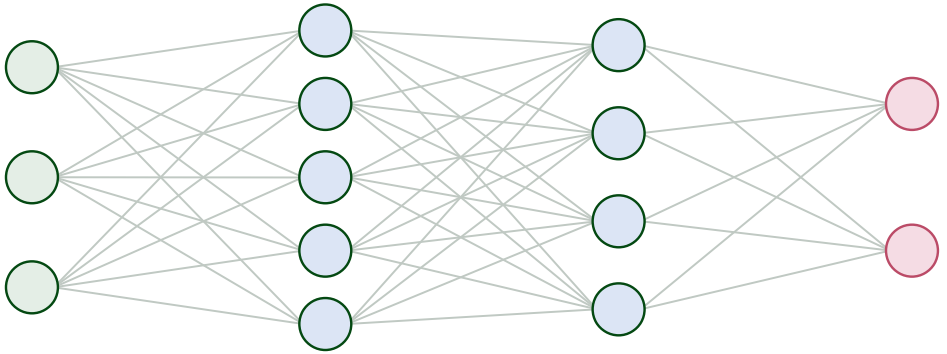
Глубокое обучение требует много данных и вычислений (GPU), поэтому стало массовым только в 2010-х.

Определение нейронной сети

Нейронная сеть — это параметризованная композиция преобразований.

Вход последовательно проходит через слои нейронов, каждый из которых умножает вход на веса, добавляет смещение и применяет функцию активации.

Параметры θ — это веса и смещения всех слоёв. Обучение подбирает θ так, чтобы выход сети совпадал с правильными ответами.



$$f_\theta(x) = f^L(\dots f^2(f^1(x)) \dots)$$

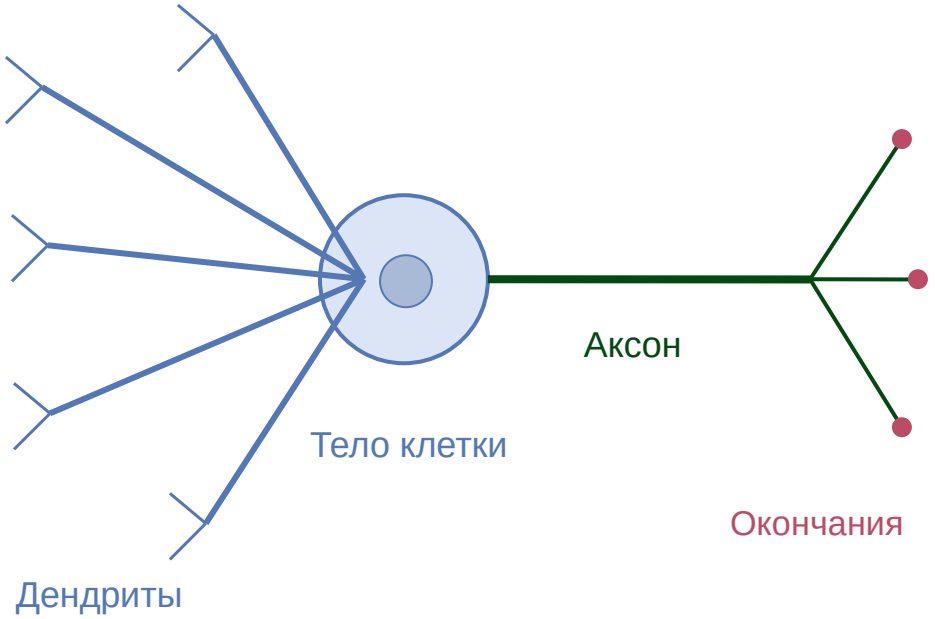
Биологический нейрон

Дендриты получают сигналы.

Тело клетки интегрирует входы.

Аксон проводит импульс к другим клеткам через синапсы.

Биологическая динамика значительно сложнее искусственной модели.



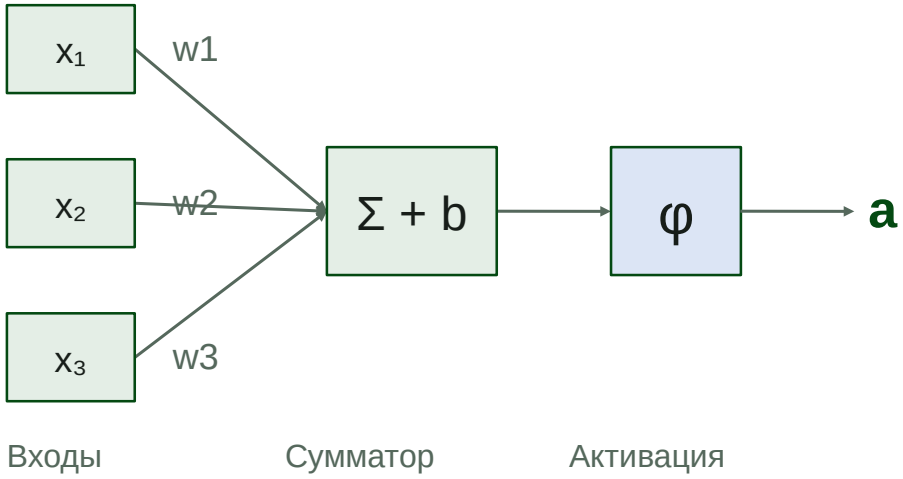
Искусственный нейрон

Входы x_j умножаются на веса w_j.

Сумматор добавляет смещение b.

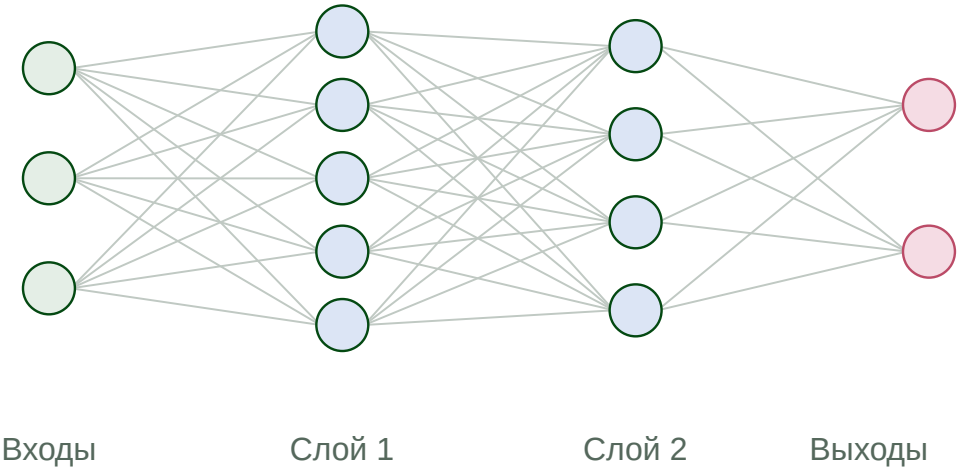
Функция активации φ формирует выход a.

$z = \Sigma_j w_j x_j + b$
$a = \varphi(z)$



Архитектура базовой сети

Входной слой принимает признаки объекта.
Скрытые слои последовательно преобразуют их в промежуточные представления.
Выходной слой выдает результат, соответствующий задаче: класс, вероятность или число.



$$h^{(l)} = \varphi(W^{(l)}h^{(l-1)} + b^{(l)})$$

Классификация нейронных сетей

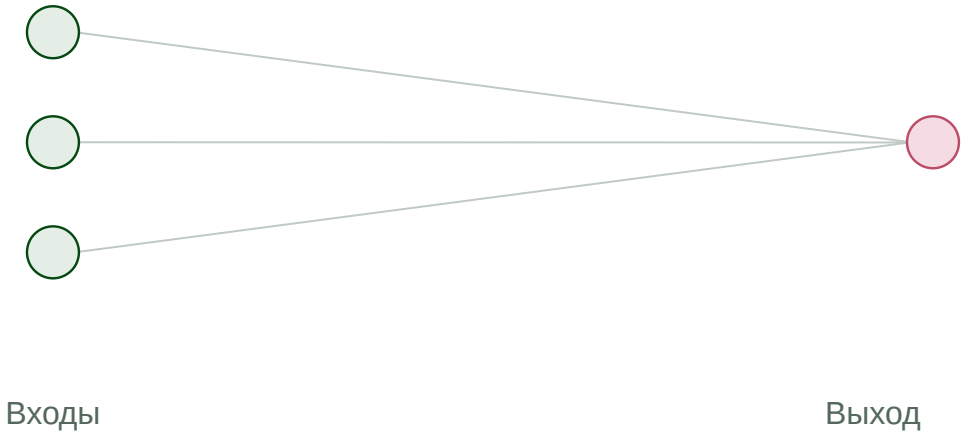
Основание	Варианты
Направление связей	Прямое распространение, рекуррентные
Операции	Полносвязные, свертки, внимание
Обучение	С учителем, без учителя, self-supervised, RL

Однослойный перцептрон

Один обучаемый слой вычисляет взвешенную сумму и пороговый ответ.

Граница решения линейная.

Линейно неразделимую функцию XOR такой перцептрон не решает.



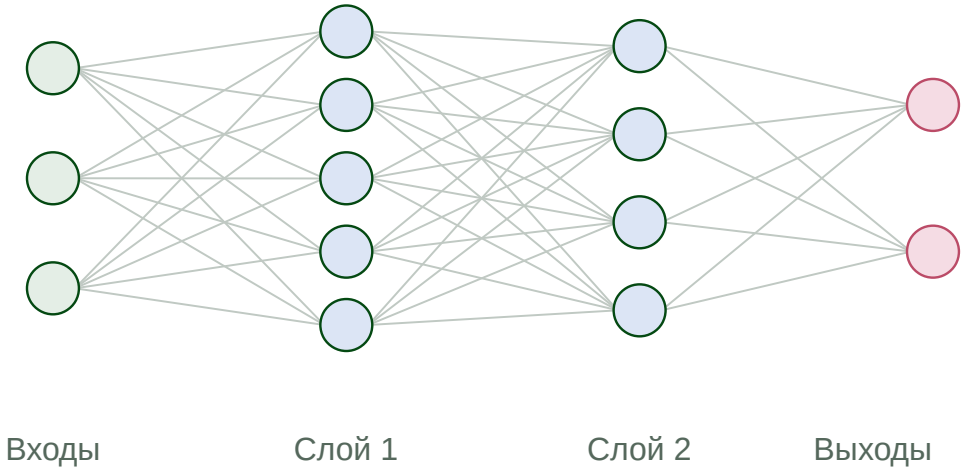
$$\hat{y} = 1[w^T x + b \geq 0]$$

Многослойный перцептрон: MLP

Скрытые слои с нелинейными активациями создают сложные границы.

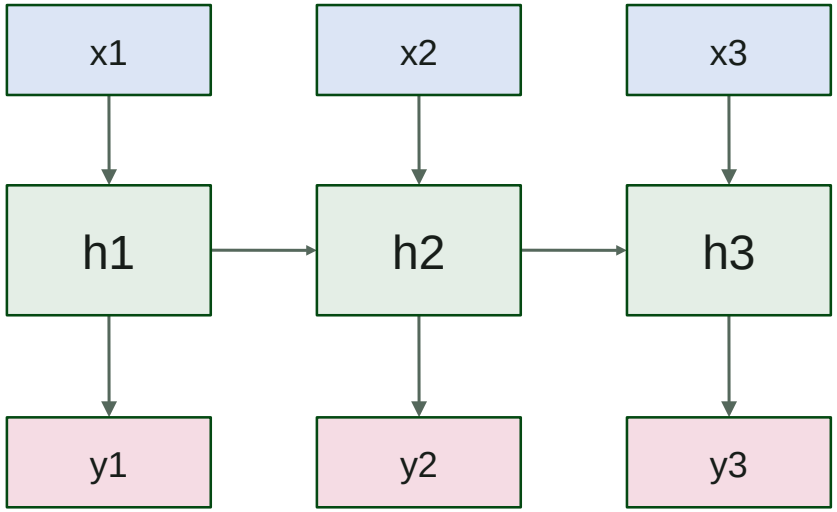
Все связи направлены от входа к выходу.

MLP используют для табличных признаков и как блок других архитектур.



Рекуррентная нейронная сеть: RNN

Скрытое состояние хранит информацию о предыдущих шагах. Одни и те же параметры применяются вдоль последовательности.
LSTM и GRU используют механизмы управления памятью.



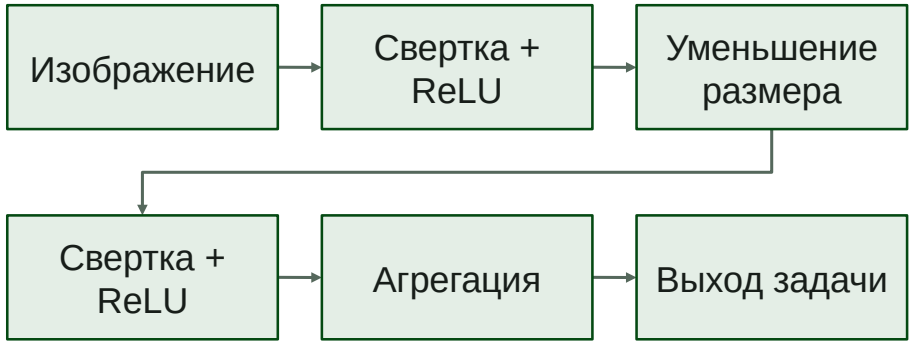
Общие параметры на каждом шаге

$$h_t = \varphi(W_x x_t + W_h h_{t-1} + b)$$

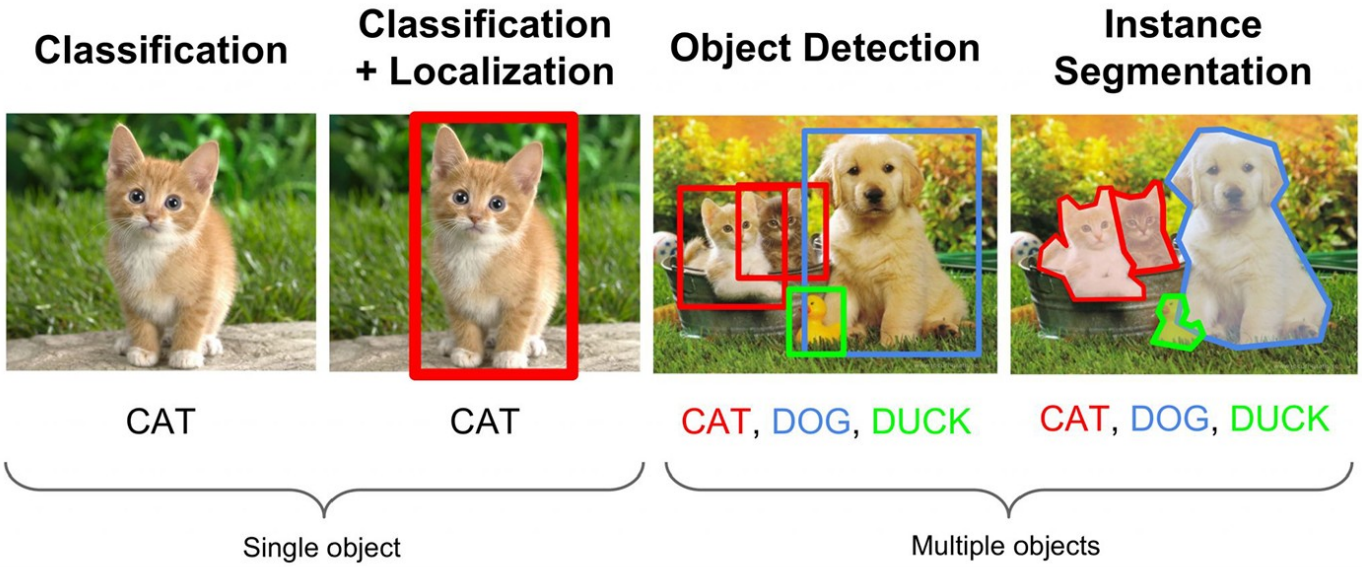
Источник: RNN · ссылки в заметках

Сверточная нейронная сеть

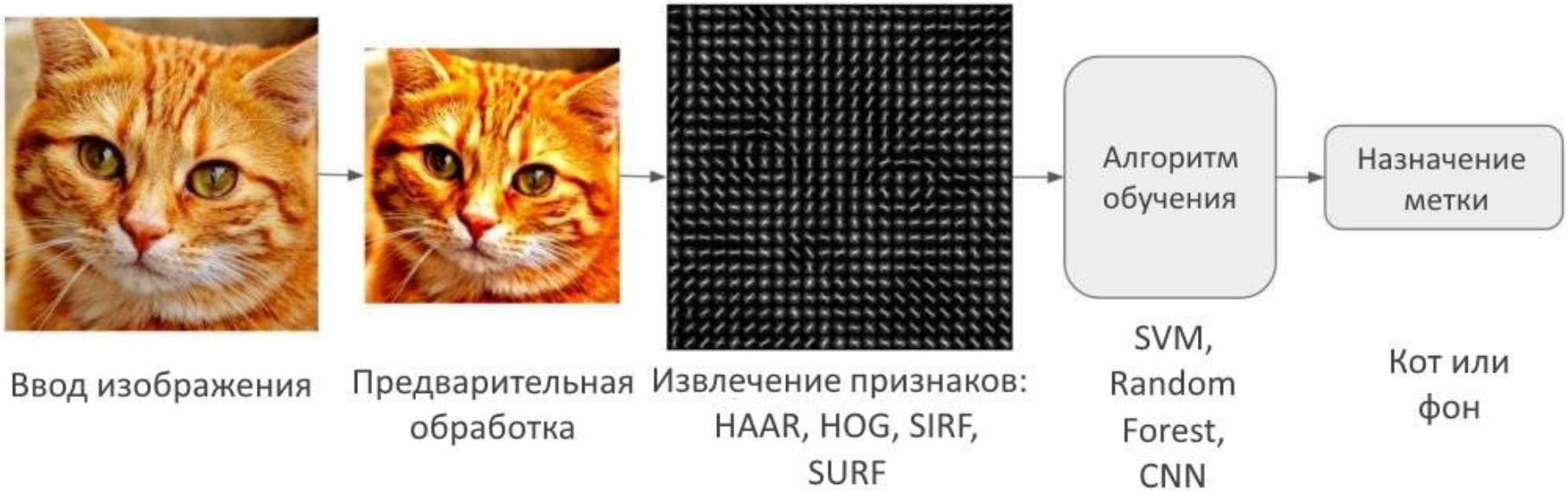
Свертка применяет локальное ядро с общими весами ко всему изображению. Результат — карта признаков, которая показывает отклик фильтра. Слои последовательно формируют пространственные представления: от простых (края, линии) к сложным (объекты). Изображение проходит через череду сверток и нелинейностей. Уменьшение пространственного размера объединяет локальную информацию. Выходной слой зависит от задачи: классификация, детекция, сегментация.



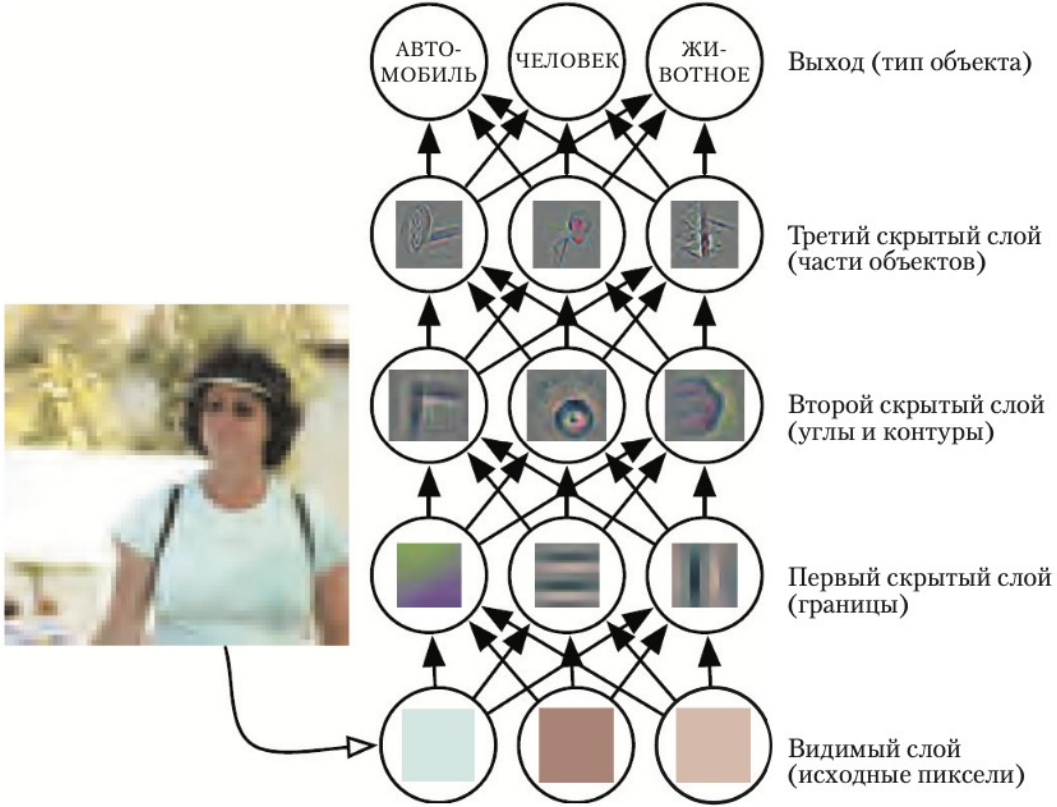
Задачи компьютерного зрения



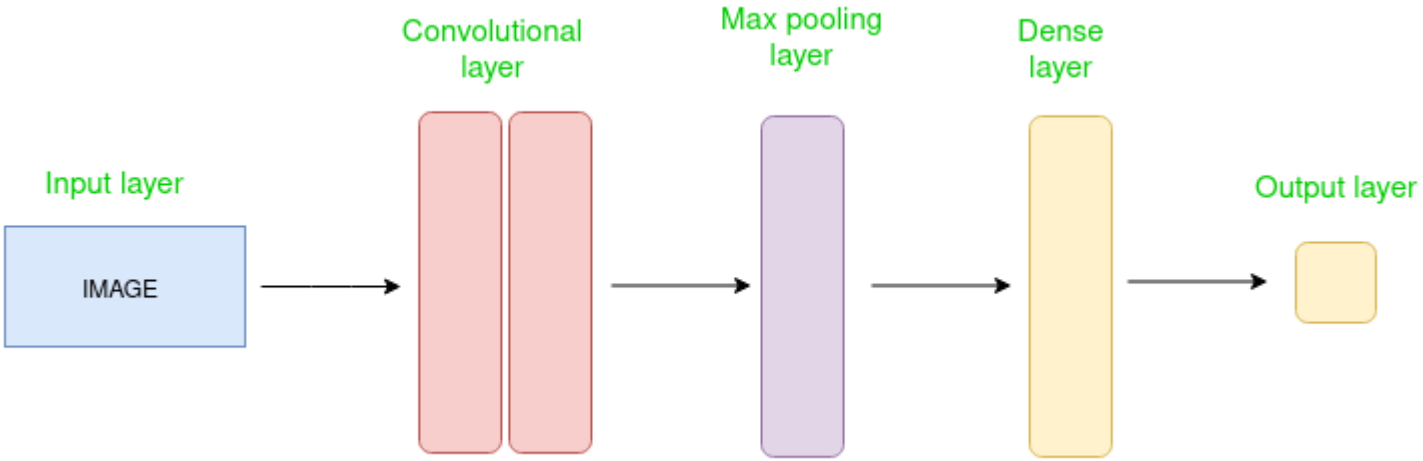
Распознавание объекта на изображении



Нейронная сеть для распознавания образов



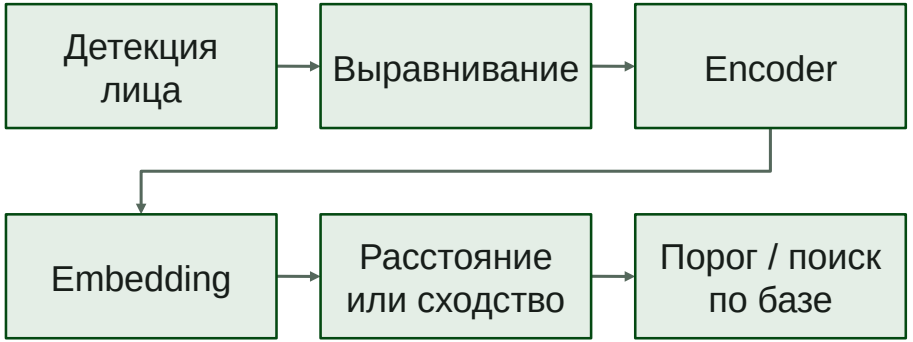
Сверточная нейронная сеть



Распознавание лиц

Детекция находит лицо.

Сеть преобразует лицо в вектор признаков: embedding.
Верификация сравнивает пару лиц.
Идентификация ищет человека в базе.



Трансформер

Self-attention позволяет учитывать связи между элементами последовательности.
Блок включает внимание, MLP, нормализацию и остаточные связи.
Позиционная информация задает порядок элементов.

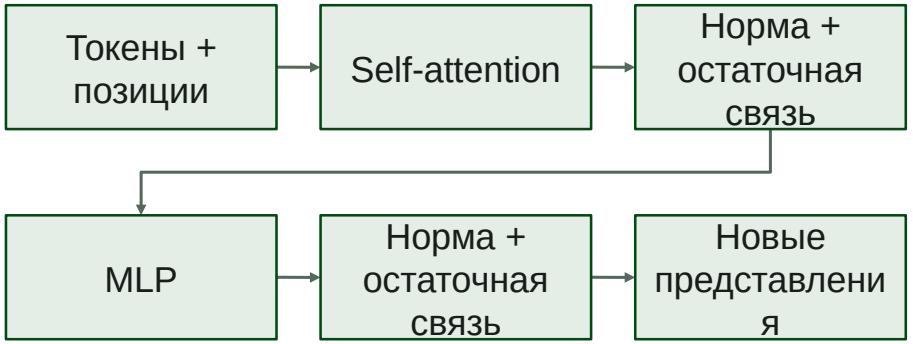


Схема одного блока, без деталей реализации

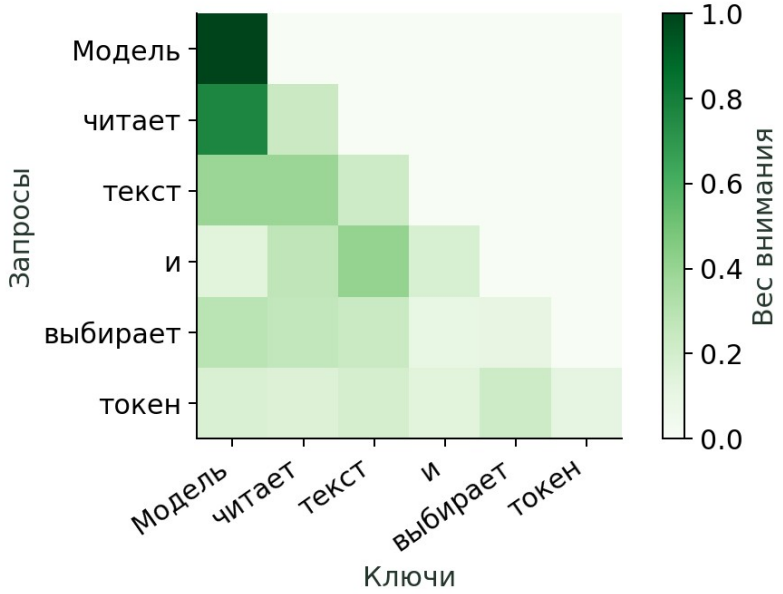
$$\mathbf{Attention(Q,K,V) = softmax(QK^T / \sqrt{d_k})\, V}$$

Внимание и причинная маска

В генерации токен может учитывать только предыдущие токены и себя.

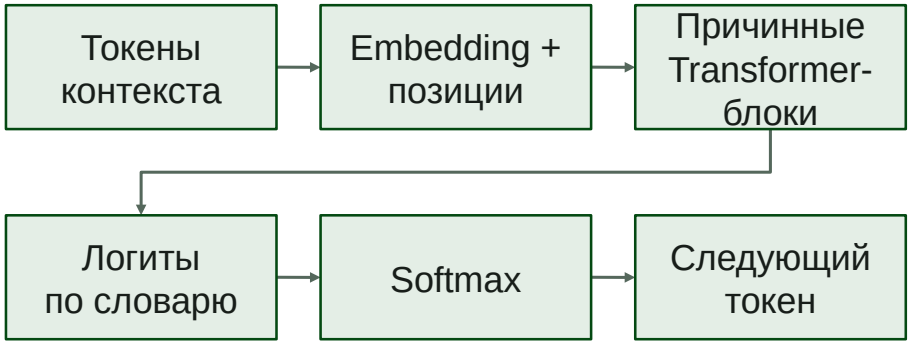
Будущие позиции маскируются.

Разные головы внимания обучают разные преобразования.



Большие языковые модели: пример GPT

GPT 2: авторегрессионная модель на основе decoder-only Transformer.
Предобучение предсказывает следующий токен по контексту.
Токен может быть словом, частью слова или символом.



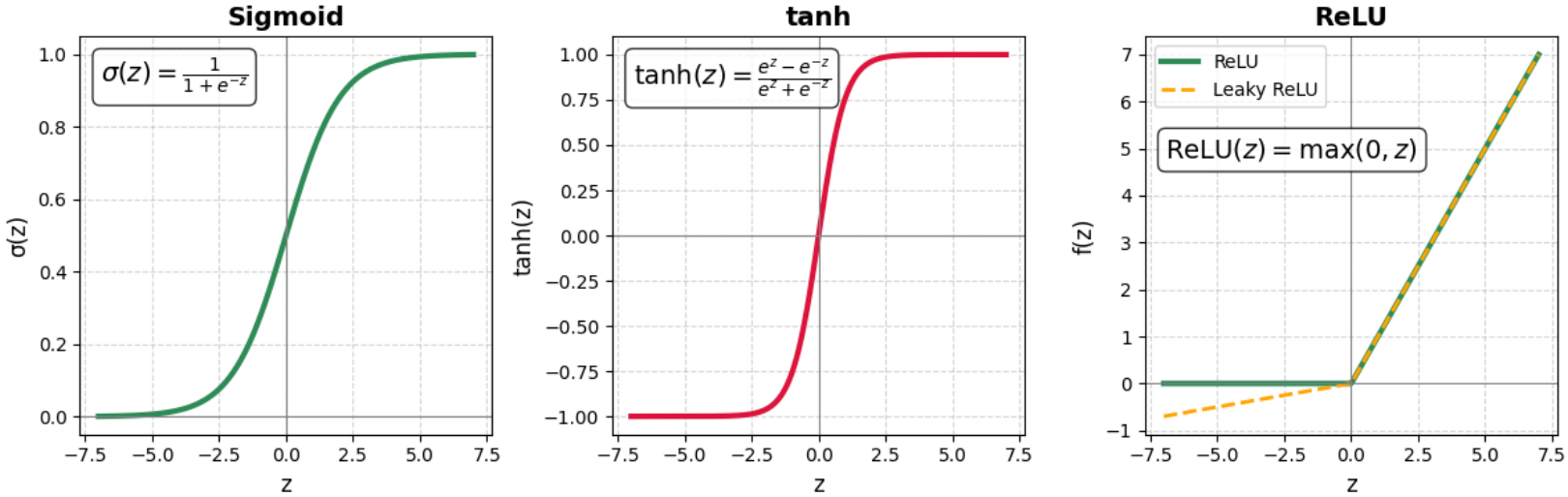
$$P(t_1,\ldots,t_T) = \prod_i P(t_i \mid t_1,\ldots,t_{i-1})$$

Большие языковые модели: пример GPT



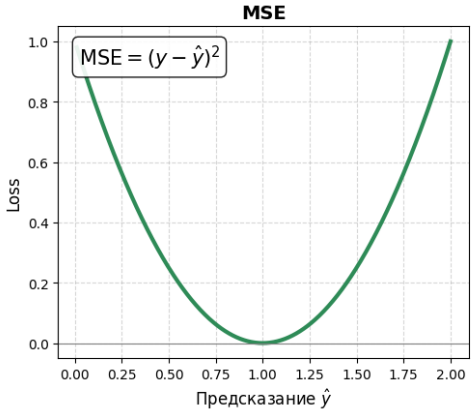
Функции активации

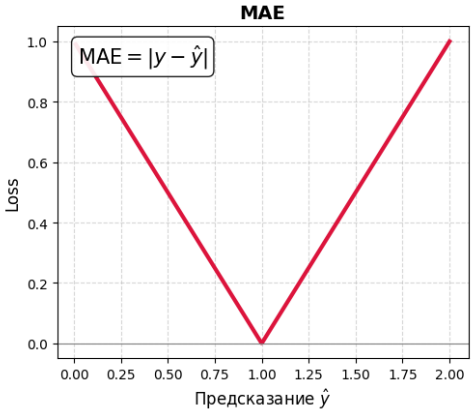
Функция активации — это нелинейное преобразование, которое применяется к выходу нейрона.
Sigmoid: выход от 0 до 1.
tanh: выход от −1 до 1.
ReLU: max(0,z).

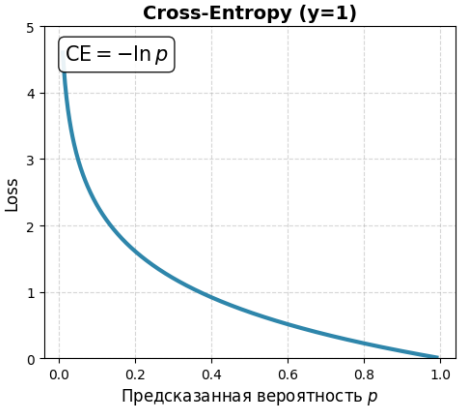


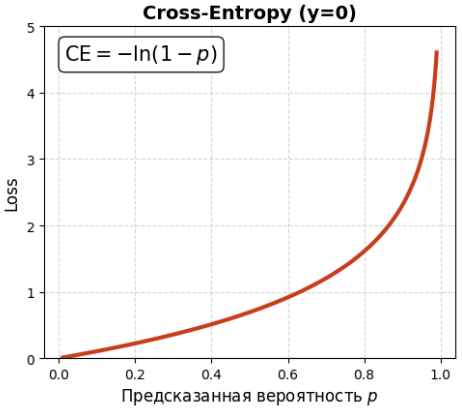
Функции потерь

Функция потерь — это мера того, насколько предсказание модели отличается от правильного ответа.
Cross-entropy: один из C классов.
MSE / MAE: регрессия и реконструкция.









Методы оптимизации

Метод оптимизации — это алгоритм, который обновляет параметры сети (веса и смещения), чтобы уменьшить функцию потерь.
Batch GD: градиент по всей обучающей выборке.
SGD: по одному объекту.
Mini-batch SGD: по небольшому пакету.
Momentum, RMSprop, Adam используют историю градиентов.

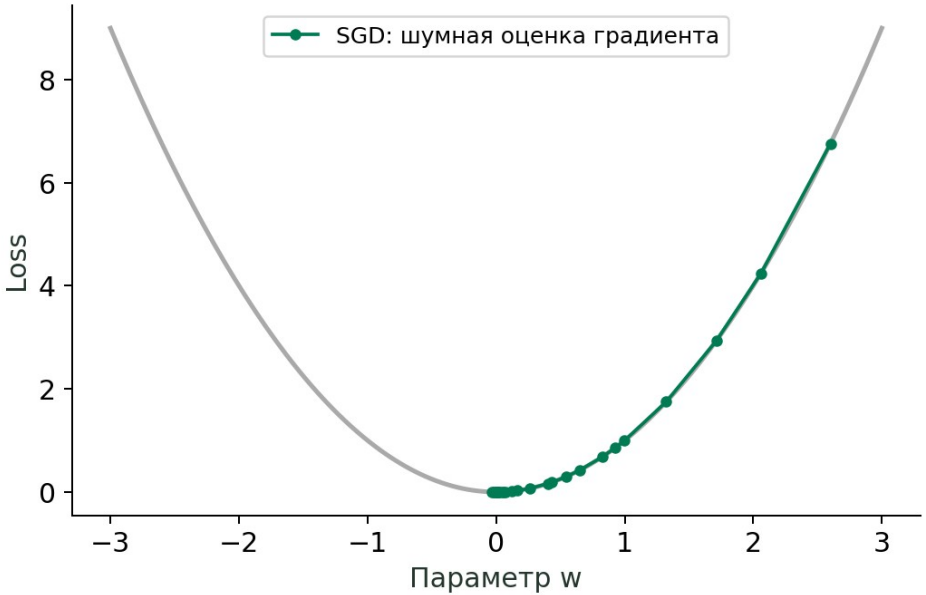
Градиентный спуск: $\theta_{t+1} = \theta_t - \eta \nabla\theta L(\theta_t)$

Стохастический градиентный спуск

На шаге берем случайный mini-batch B (небольшая часть обучающей выборки, которая подается в сеть за один шаг обучения).

Средний градиент пакета приближает градиент всей выборки.

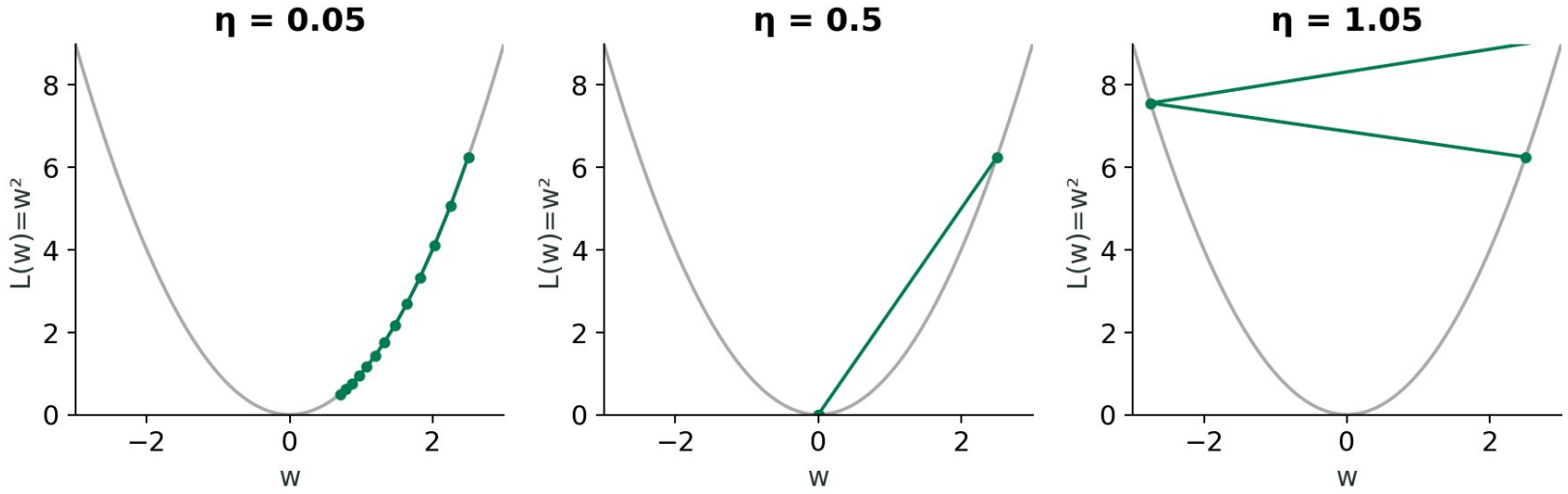
Обновляем веса и переходим к следующему пакету.



$$g^B = (1/|B|)\ \Sigma_{i \in B}\ \nabla\theta\ \ell_i(\theta)$$
$$\theta \leftarrow \theta - \eta g^B$$

Скорость обучения

Малый η: медленное движение к минимуму.
Оптимальный η: устойчивое снижение потерь.
Слишком большой η: колебания и расходимость.



Типы обучения нейронных сетей

С учителем: признаки и метки.

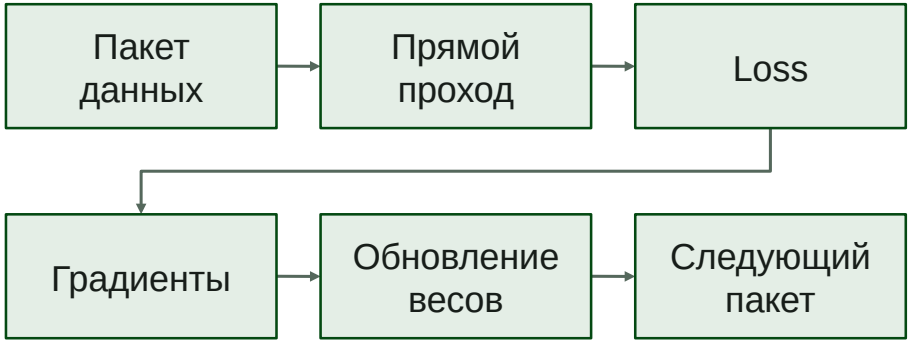
Без учителя: структура или реконструкция.

Self-supervised (самообучение): целевой сигнал строится из самих данных.

С подкреплением: действия оценивает награда среды.

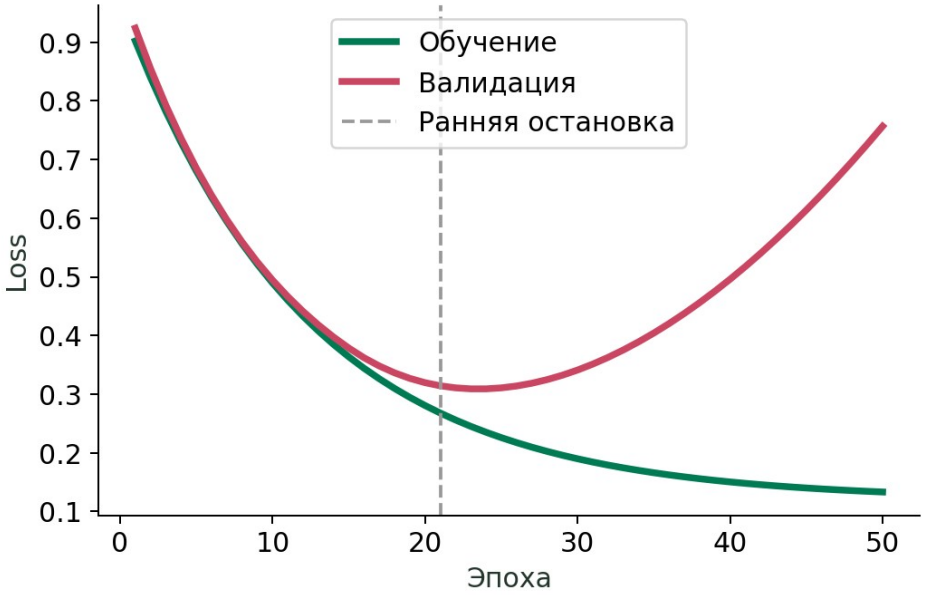
Процесс обучения сети

1. Прямой проход: вычисляем прогноз.
2. Считаем loss.
3. Backpropagation: вычисляем градиенты.
4. Оптимизатор обновляет параметры.
5. Повторяем по пакетам и эпохам.



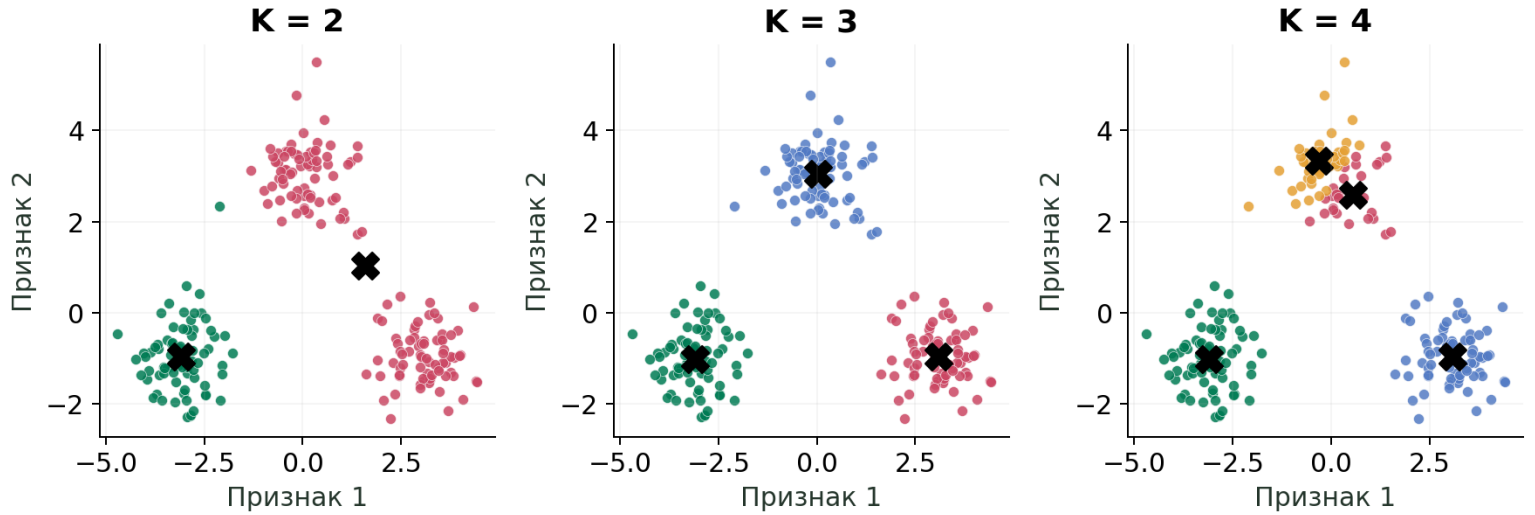
Переобучение нейронной сети

На обучении loss уменьшается, на валидации перестает улучшаться или растёт.
Причины: избыточная сложность, мало данных, шум, утечка.
Средства: early stopping, регуляризация, dropout, аугментации.



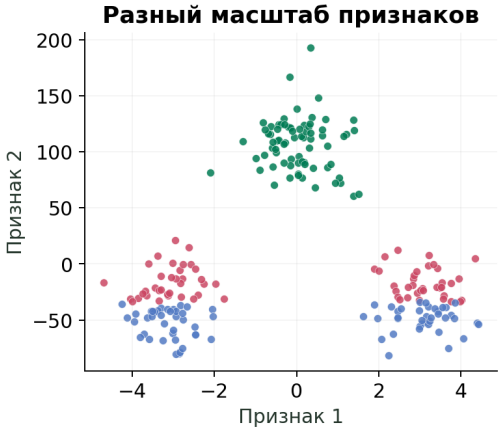
Задача кластеризации

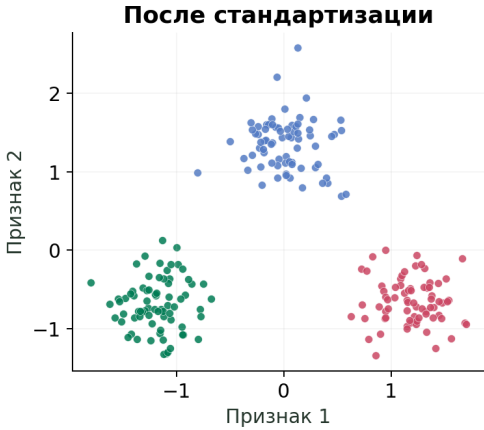
Алгоритм группирует объекты по сходству выбранных признаков.
Метки классов при обучении отсутствуют.



Условия применения кластеризации

Есть содержательные признаки и понятное определение сходства.
Нужен поиск структуры, сегментов или аномалий.
Данные подготовлены: масштаб, пропуски, выбросы.
Полученные группы можно проверить и интерпретировать.

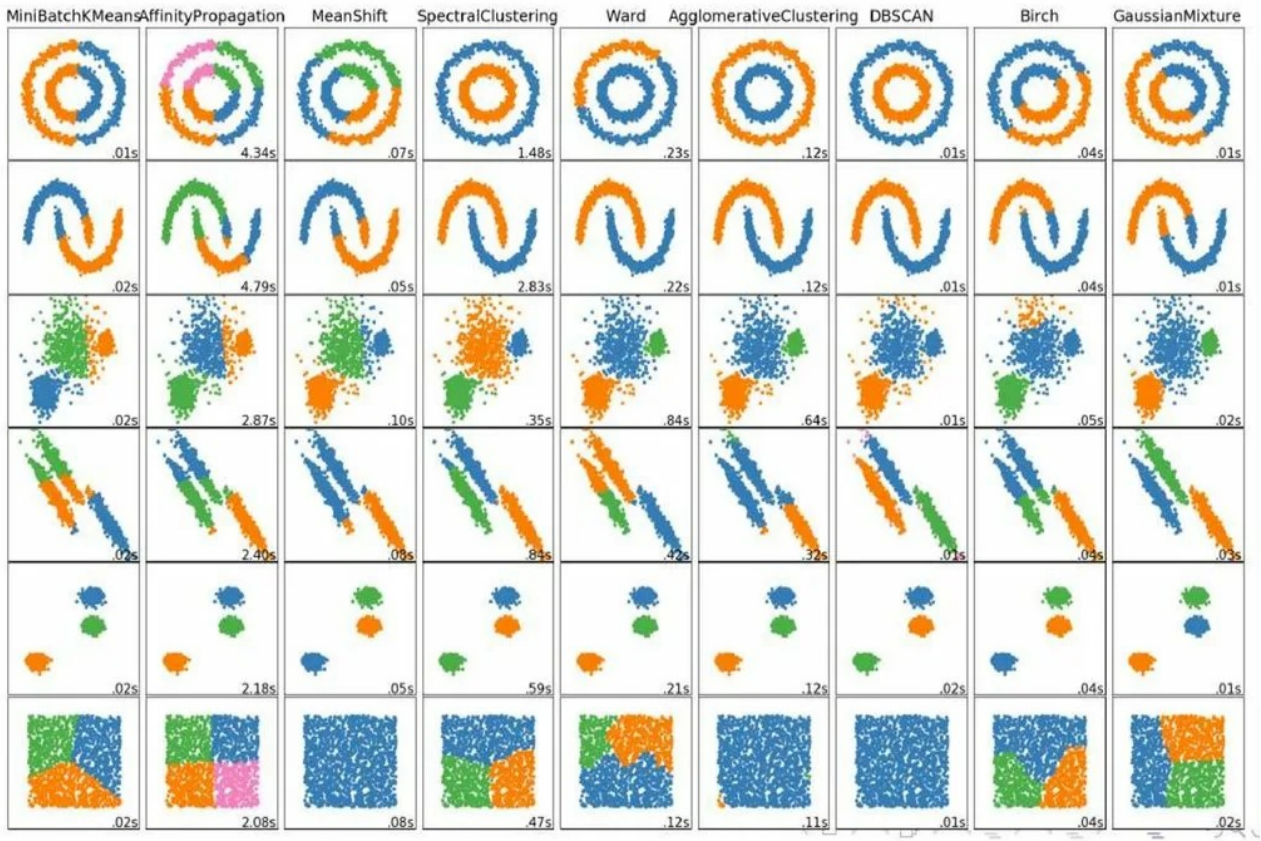




Виды методов кластеризации

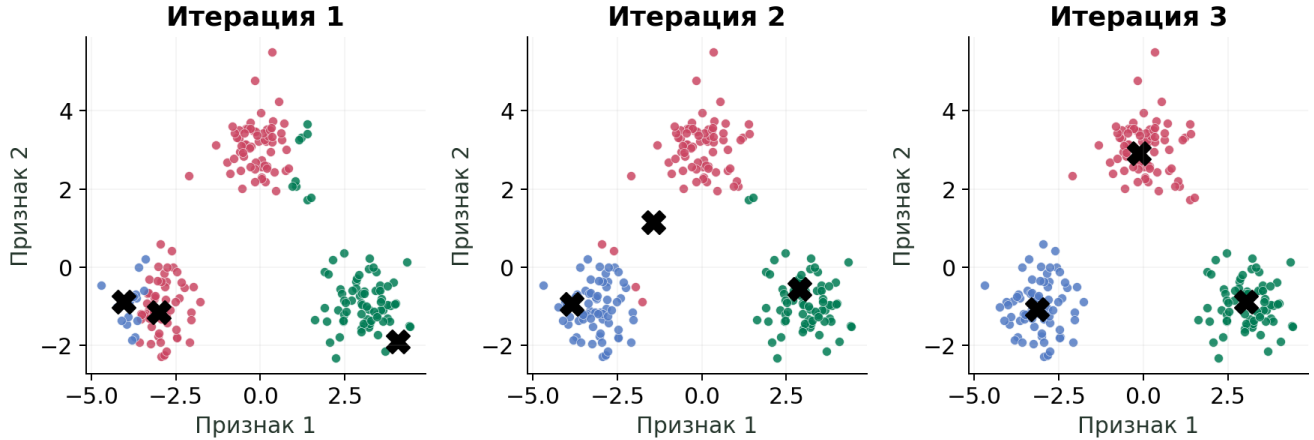
Семейство	Идея	Пример
Центроидные	Близость к центру	K-means
Иерархические	Дерево вложенных групп	Agglomerative
Плотностные	Связные плотные области	DBSCAN
Вероятностные	Смесь распределений	GMM
Представления	Обучение embedding	DEC, VaDE, DeepCluster
Конкурентные	Топологическая карта	SOM

Виды методов кластеризации



K-means

Каждый объект относится к ближайшему центру.
Центр кластера равен среднему его объектов.
Минимизируем сумму квадратов евклидовых расстояний.

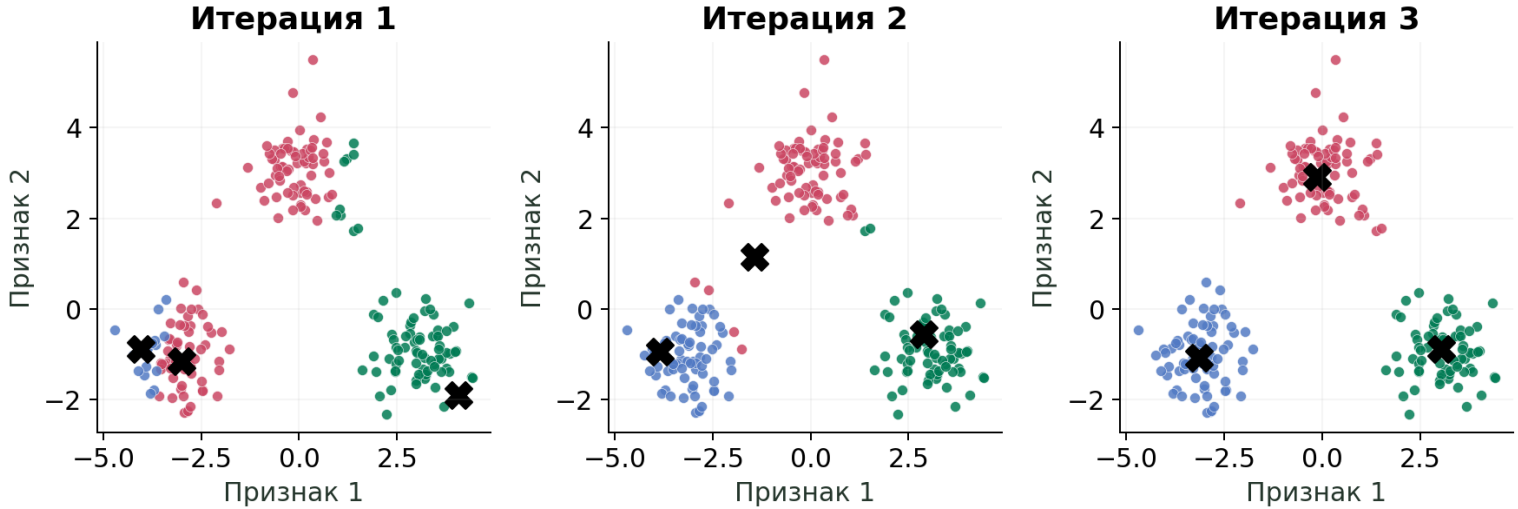


$$J = \Sigma_k \Sigma_{x \in C_k} ||x - \mu_k||^2$$

K-means: шаги и ограничения

1. Выбрать K и начальные центры.
2. Назначить ближайший центр каждому объекту.
3. Пересчитать центры.
4. Повторять до остановки.

Чувствителен к выбросам и масштабу.

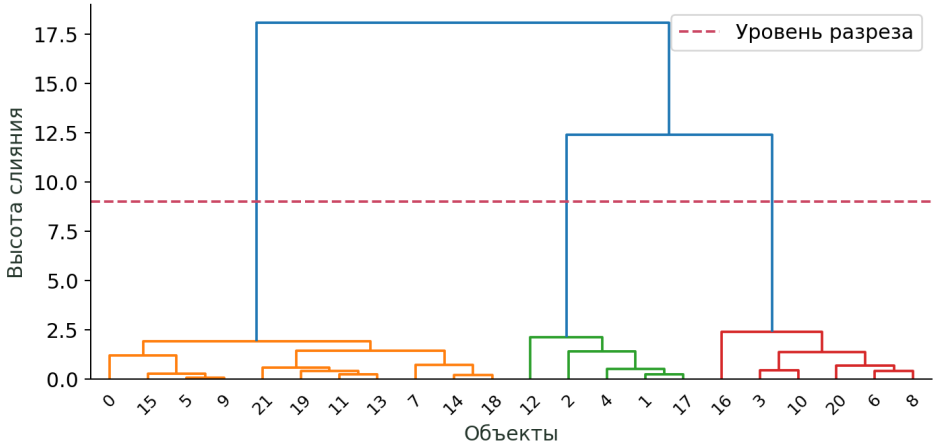


Иерархическая кластеризация

Агломеративный алгоритм начинает с отдельных объектов.

На каждом шаге объединяет ближайшие группы.

Дендрограмма показывает порядок и высоты слияний.



Методы вычисления расстояния между кластерами

Одиночная связь (Single Linkage)

Минимальное расстояние между точками кластеров.
Склонен к цепному эффекту.

$$d(A,B) = \min\{d(a,b) : a \in A, b \in B\}$$

Полная связь (Complete Linkage)

Максимальное расстояние между точками кластеров.
Даёт компактные кластеры.

$$d(A,B) = \max\{d(a,b) : a \in A, b \in B\}$$

Средняя связь (Average Linkage)

Среднее всех попарных расстояний.
Компромиссный вариант.

$$d(A,B) = \frac{1}{|A||B|} \sum_{a \in A} \sum_{b \in B} d(a,b)$$

Центроидный метод

Расстояние между центроидами кластеров.

$$d(A,B) = \|\mu_A - \mu_B\|$$

Медианный метод

Центр нового кластера — середина между центрами.
Уравнивает влияние частей.

$$\mu_{A \cup B} = \frac{\mu_A + \mu_B}{2}$$

Метод Варда (Ward)

Минимизирует прирост суммы квадратов
расстояний до центров.

$$\Delta = \sum_{x \in A \cup B} \|x - \mu_{A \cup B}\|^2 - \sum_{x \in A} \|x - \mu_A\|^2 - \sum_{x \in B} \|x - \mu_B\|^2$$

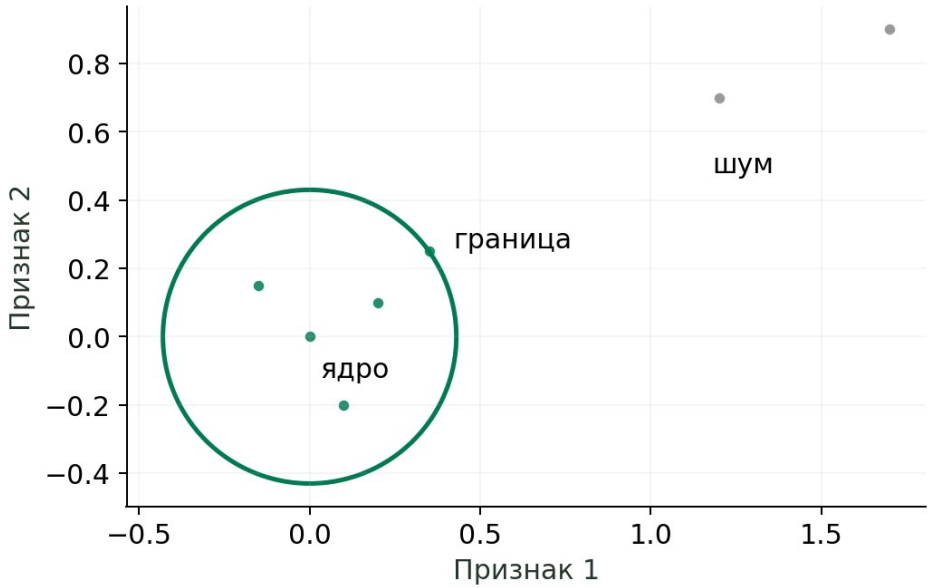
DBSCAN: плотность и шум

ε задает радиус окрестности.

min_samples задаёт минимальную плотность, включая саму точку.

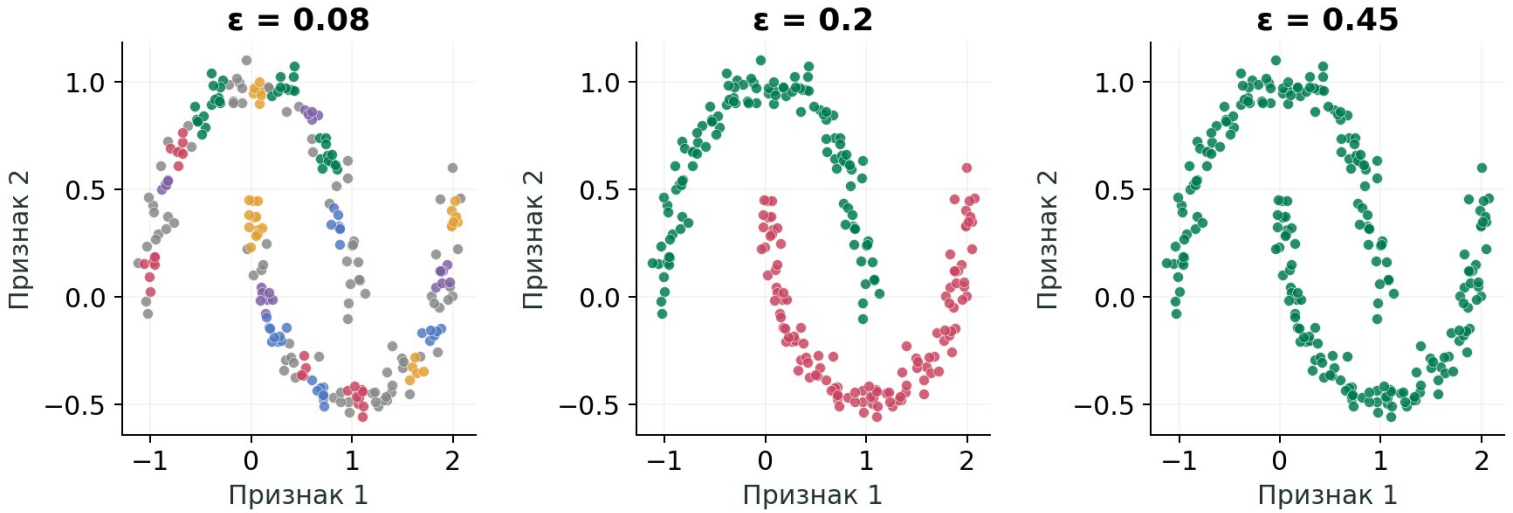
Ядра соединяют плотные области.

Шум получает метку −1.



DBSCAN: выбор параметров

Малое ε: много шума и мелких групп.
Большое ε: группы могут сливаться.
Разная плотность групп усложняет подбор.

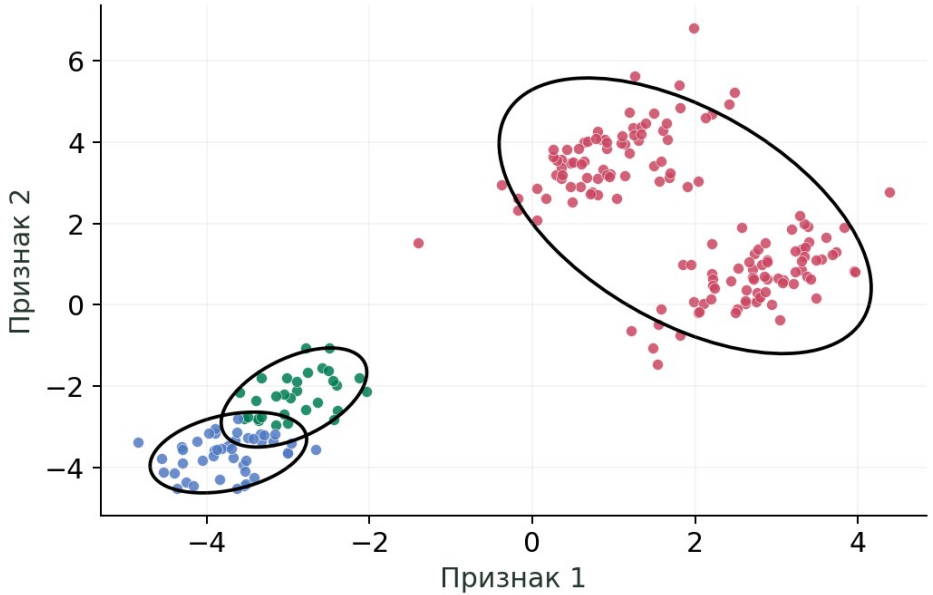


Gaussian Mixture: мягкая кластеризация

GMM описывает данные смесью нормальных компонент.

Компонента имеет вес π_k, среднее μ_k и ковариацию Σ_k.

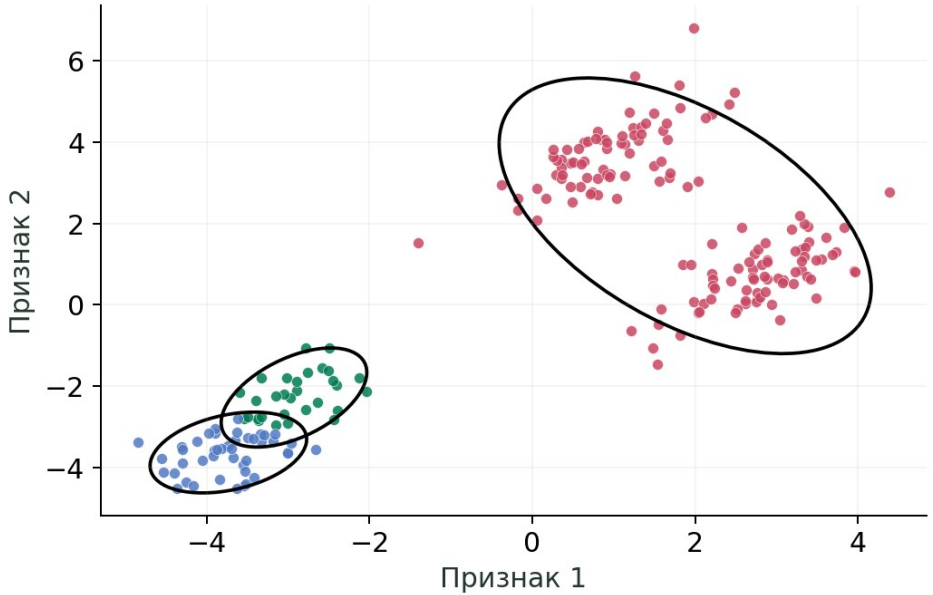
Объект получает вероятности принадлежности.



$$p(x) = \Sigma_k \pi_k N(x \mid \mu_k, \Sigma_k), \quad \Sigma_k \pi_k = 1$$

EM-алгоритм для GMM

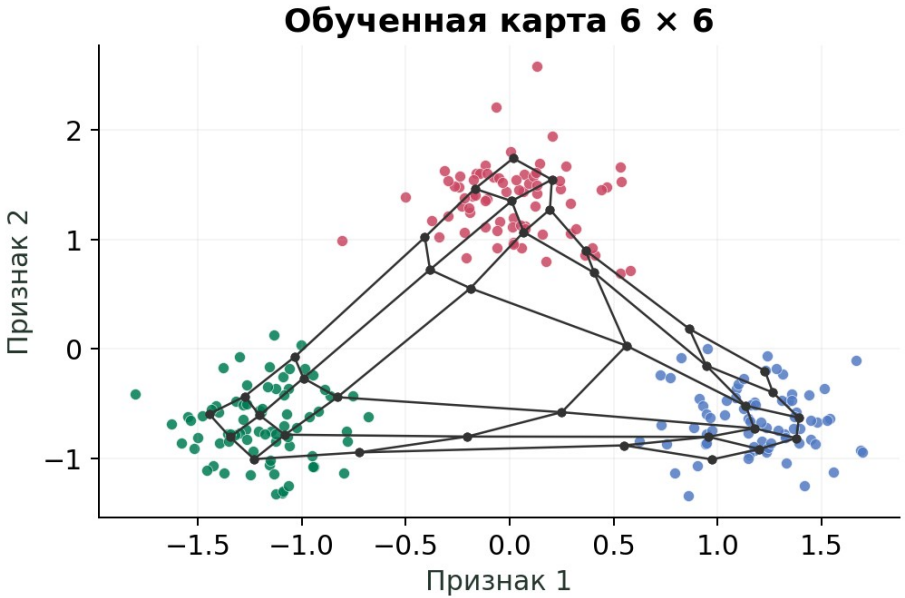
E-шаг: вычислить ответственности компонент r_{ik}.
M-шаг: пересчитать параметры по взвешенным объектам.
Повторять до стабилизации правдоподобия.



$$r_{ik} = \pi_k \, N(x_i \mid \mu_k, \Sigma_k) \,/\, \Sigma_j \, \pi_j \, N(x_i \mid \mu_j, \Sigma_j)$$

SOM: самоорганизующаяся карта

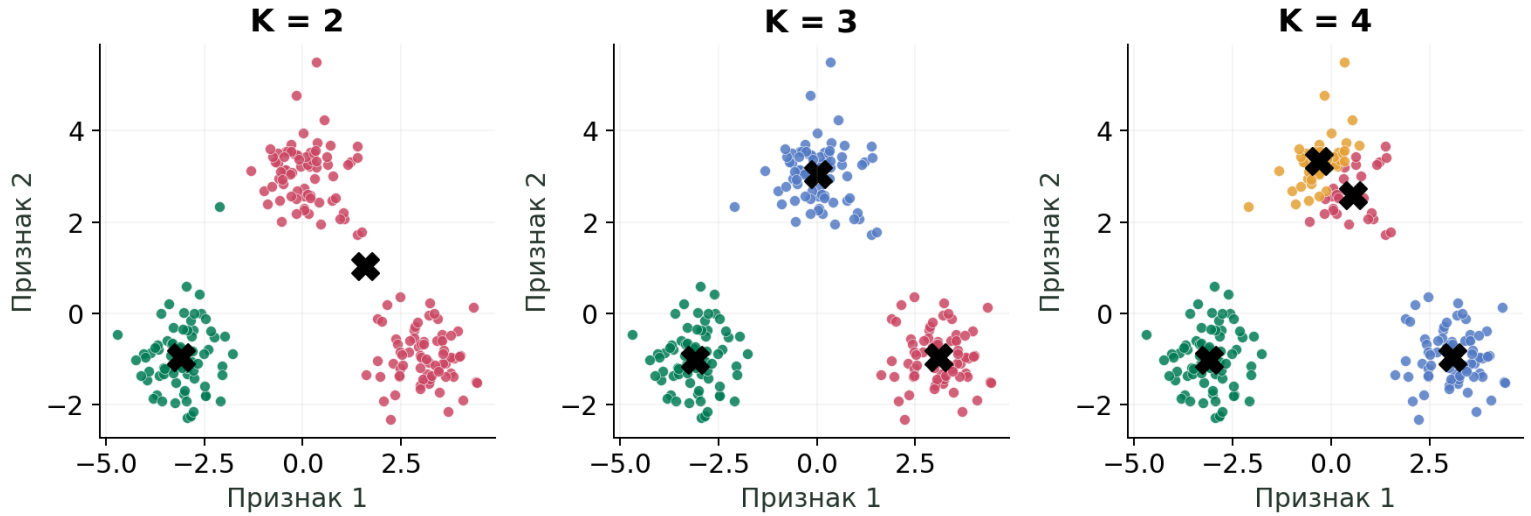
Нейроны образуют решётку прототипов.
Победитель BMU (нейрон-победитель) ближе всего к объекту.
Обновляются победитель и его соседи на карте.
SOM помогает изучать топологию данных.



$$w_j \leftarrow w_j + \eta\, h_{j,BMU}\,(x - w_j)$$

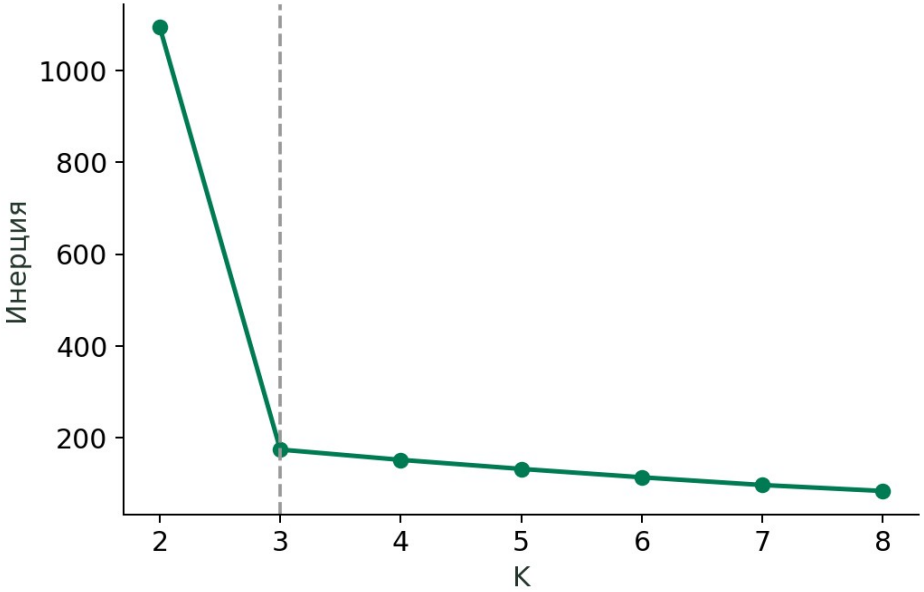
Число кластеров: диаграмма рассеивания

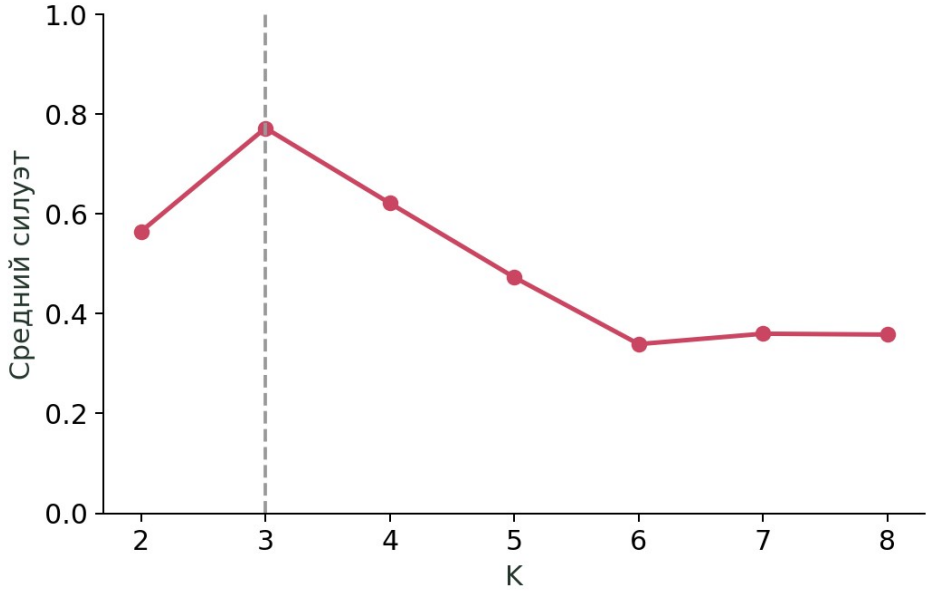
На двумерном рисунке можно оценить разделение, выбросы и форму групп.
Сравнение нескольких K помогает заметить слияние или дробление.
Проекция может скрывать структуру исходного пространства.



Метод локтя и silhouette

Инерция K-means обычно уменьшается при росте K.
Локоть: заметное замедление улучшения.
Silhouette сравнивает близость внутри группы и к ближайшей другой группе.





Меры расстояния между точками

Евклидово:

$$d_2(x, y) = \sqrt{\sum_{j=1}^{p} (x_j - y_j)^2}$$

Манхэттенское:

$$d_1(x, y) = \sum_{j=1}^{p} |x_j - y_j|$$

Косинусное:

$$d_{\cos}(x, y) = 1 - \frac{x^T y}{\|x\| \cdot \|y\|}$$

Махаланобиса:

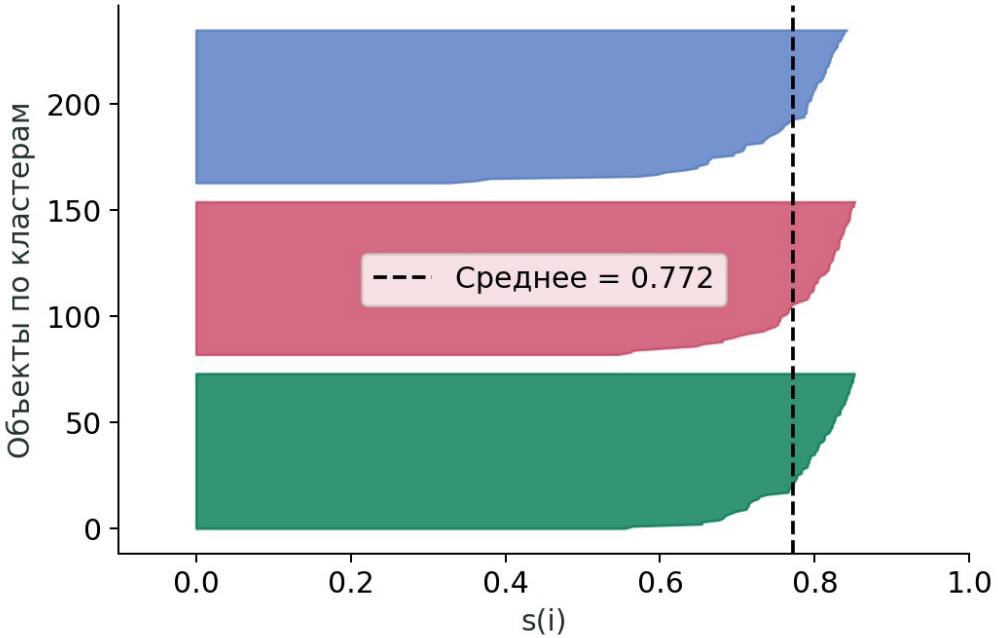
$$d_M(x, y) = \sqrt{(x - y)^T \Sigma^{-1} (x - y)}$$

Jaccard (для множеств):

$$d_J(A, B) = 1 - \frac{|A \cap B|}{|A \cup B|}$$

Внутренние метрики
Silhouette: формула и расчёт

Эталонные метки не нужны.
Silhouette: выше лучше.
Calinski–Harabasz: выше лучше.
Davies–Bouldin: ниже лучше.
Метрика отражает выбранную геометрию, а не смысл групп.
s(i) = [b(i) − a(i)] / max(a(i),b(i))
a(i): среднее расстояние до других объектов своего кластера.
b(i): минимальное среднее расстояние до другого кластера.
От −1 до 1: выше означает лучшее разделение.



Calinski–Harabasz и Davies–Bouldin

CH сравнивает межкластерный и внутрикластерный разброс с поправкой на K и n.

DB усредняет худшее отношение разброса к расстоянию между центрами.

$$CH = [B/(K-1)] / [W/(n-K)]$$

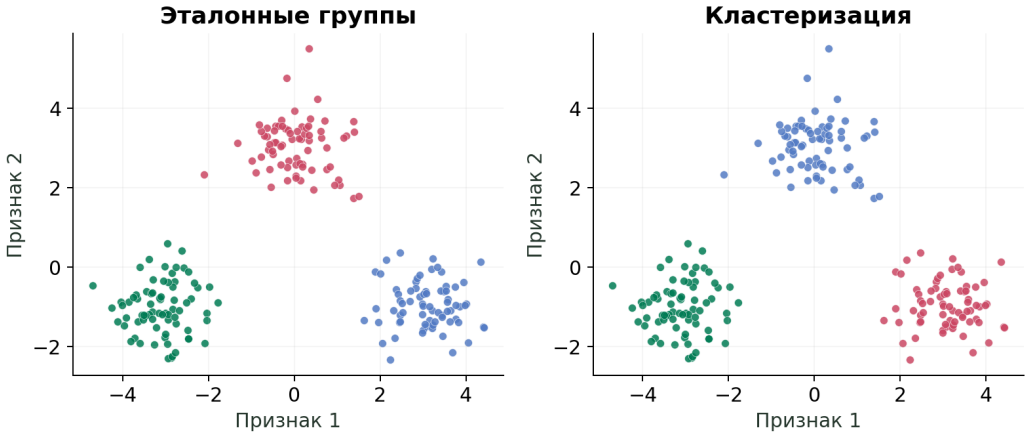
$$DB = (1/K)\ \Sigma_i \max_{j \neq i} [(S_i+S_j)/M_{ij}]$$

Внешние метрики кластеризации

Есть эталонное разбиение для проверки.

Сравнивают структуру групп, а не номера кластеров.

ARI, NMI, homogeneity, completeness и V-measure.



Rand Index и Adjusted Rand Index

RI оценивает согласованность пар объектов: вместе или отдельно в обоих разбиениях.

ARI корректирует RI на случайное совпадение.

ARI=1: полное совпадение с точностью до перестановки меток.

RI = (a+b) / [n(n−1)/2]
ARI = (RI − E[RI]) / (1 − E[RI])

NMI и V-measure

NMI нормирует взаимную информацию между эталоном U и кластерами V.

Homogeneity: в кластере один эталонный класс.
Completeness: класс собран в один кластер.
V-measure: их гармоническое среднее.

NMI = 2 I(U;V) / [H(U)+H(V)]
V = 2hc / (h+c)